# SOURCE CODE GUIDE

**Frapper** is completed project template with all features which are commonly used in projects.

**Technologies** ASP.NET Core 3.1, Entity Framework Core 3.1, jQuery, Bootstrap, SQL Server

**Feature Overview**

- Alerts
- Custom User Management
- Unit Of Work Using Entity Framework
- Unit Of Work Using Dapper
- Configure Menus by Roles
- Custom Grids
- Captcha
- Cache Services
- Dapper
- Entity Framework Core
- Create Pdf files
- Create Notice
- Capture Photo from WebCam
- Cropping Images and storing
- Create Notice
- Download Files
- Download Excel Reports
- Exception Logging
- FluentValidation
- Globalization and Localization
- HealthChecks
- jQuery Datetime pickers & Bootstrap Datetime pickers
- Jtable Grid
- Loader
- Modal popups
- Notification
- Ordering Menus
- Upload Files to store in a folder
- Upload Files to store in Database

---

**NuGet Packages which are used in Project**

- AspNetCore.HealthChecks.Redis
- AspNetCore.HealthChecks.SqlServer
- AspNetCore.HealthChecks.UI
- AspNetCore.HealthChecks.UI.Client
- AspNetCore.HealthChecks.UI.InMemory.Storage
- AspNetCore.HealthChecks.UI.SqlServer.Storage
- AutoMapper.Extensions.Microsoft.DependencyInjection
- DinkToPdf
- DNTCaptcha.Core
- EPPlus
- FluentValidation.AspNetCore
- Microsoft.AspNetCore.Mvc.NewtonsoftJson
- Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation
- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Relational
- Microsoft.Extensions.Caching.Redis
- Microsoft.VisualStudio.Web.CodeGeneration.Design
- SixLabors.ImageSharp
- StackExchange.Exceptional.AspNetCore
- X.PagedList.Mvc.Core
- Microsoft.Data.SqlClient
- Dapper
- Newtonsoft.Json
- System.Linq.Dynamic.Core
- System.ComponentModel.Annotations

---

## Getting Started

- Clone code from Github: git clone https://github.com/saineshwar/Frapper
- Open solution Frapper.Web.sln in Visual Studio 2019
- There are 2 Database Scripts FrapperDB (Main Database),FrapperAuditDB (Request Audit Database) Download Database Script
- appsettings.json file update DatabaseConnection (FrapperDB Database) , AuditDatabaseConnection (FrapperAuditDB Database)
- Run Database Script which is provided
- Make Changes in ConnectionStrings, ApplicationSettings, Exceptional, RedisServer in appsettings.json file
- Build project which will restore all NuGet Packages
- Final Step Run Project

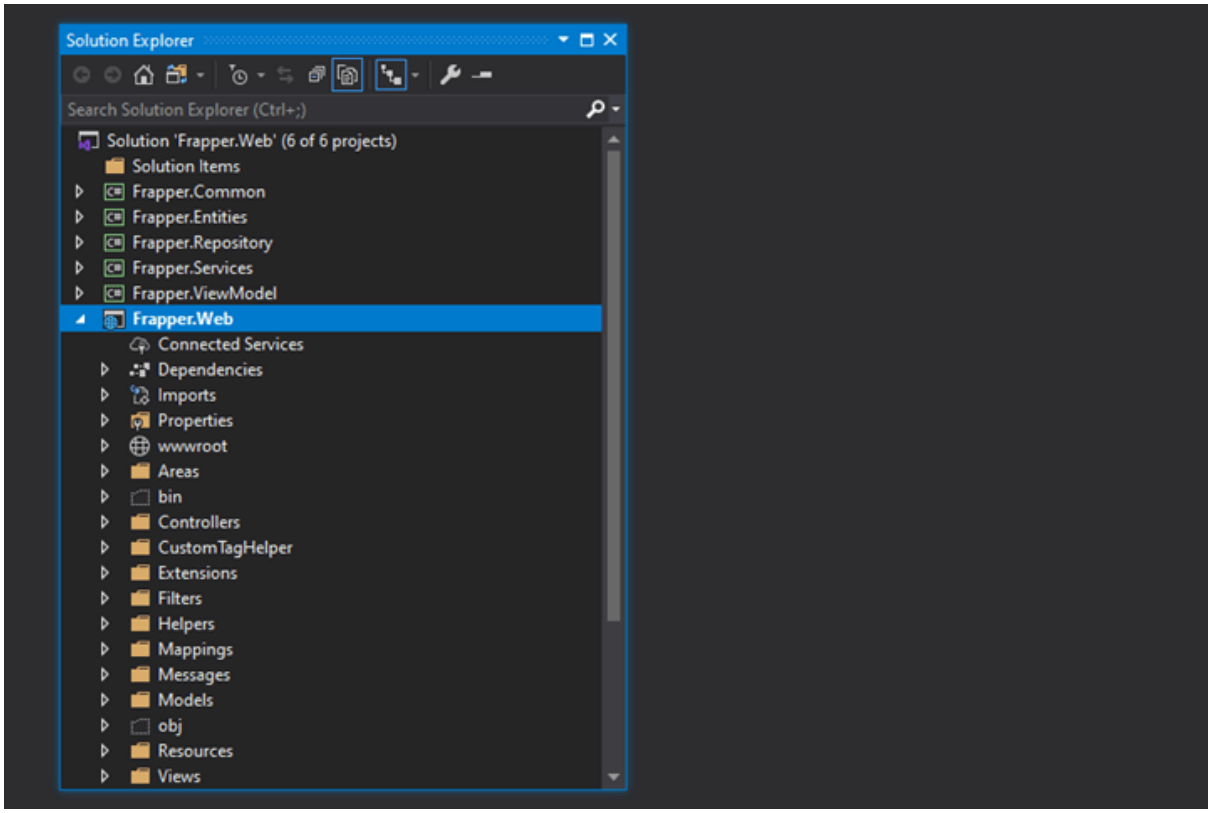
---

## Project Structure

Project Structure is simple to understand.

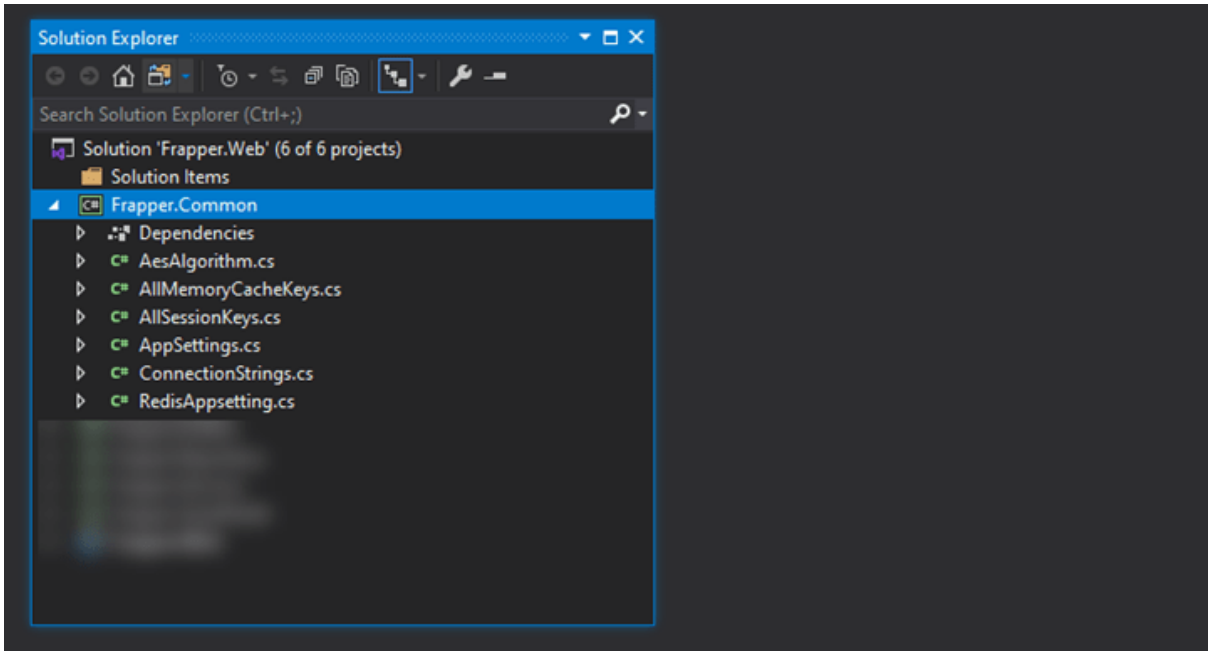
## Frapper.Web

**Frapper.Web** which is Main ASP.NET Core Project Template.



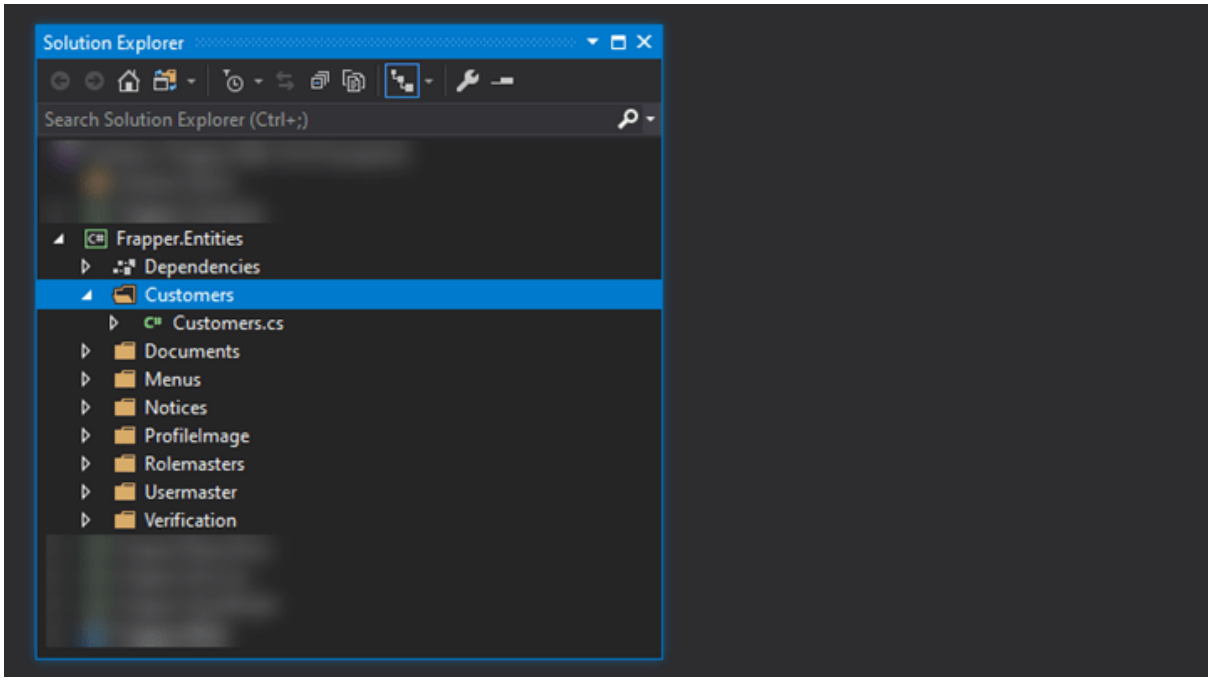
**Frapper.Common**

**Frapper.Common** this project contains all common libraries which are used in a project such as Session keys, AppSettings, ConnectionStrings, Algorithms.




**Frapper.Entities**

**Frapper.Entities** this project contains all Entities.

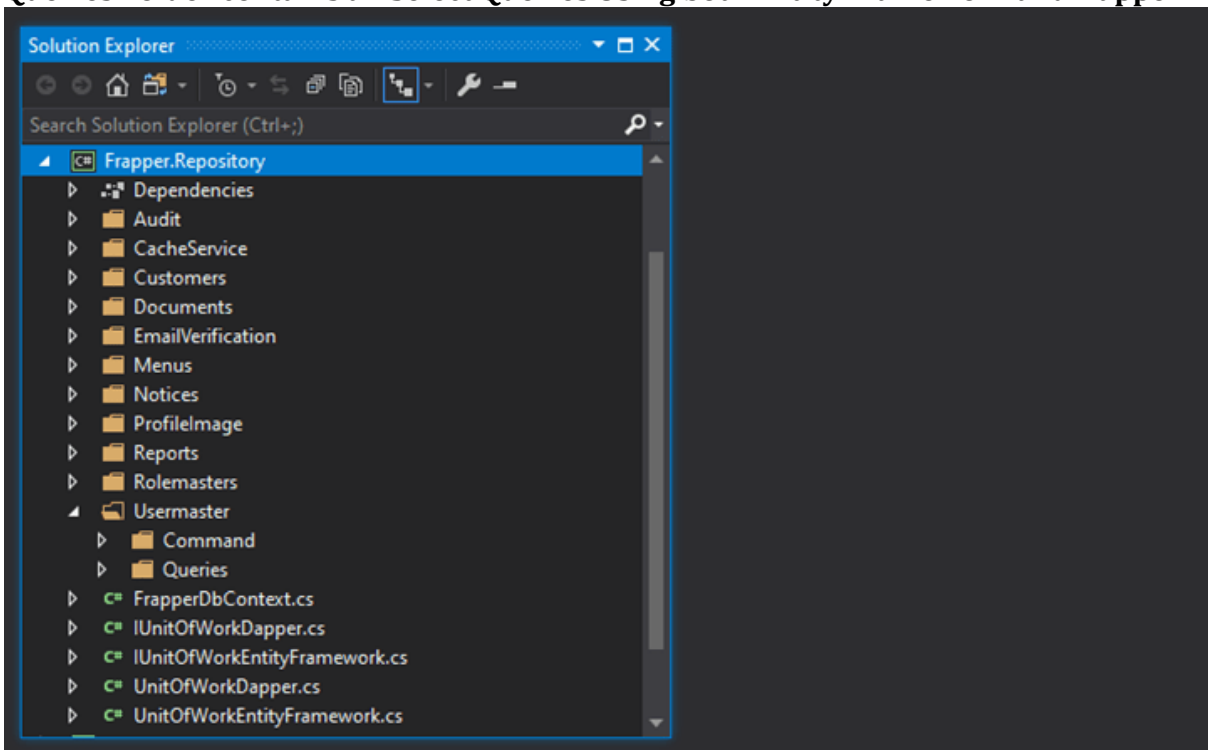


**Frapper.Repository**

**Frapper.Repository** this project contains all Class which access database. In this part we have separated Responsibility into two different part reading and writing.

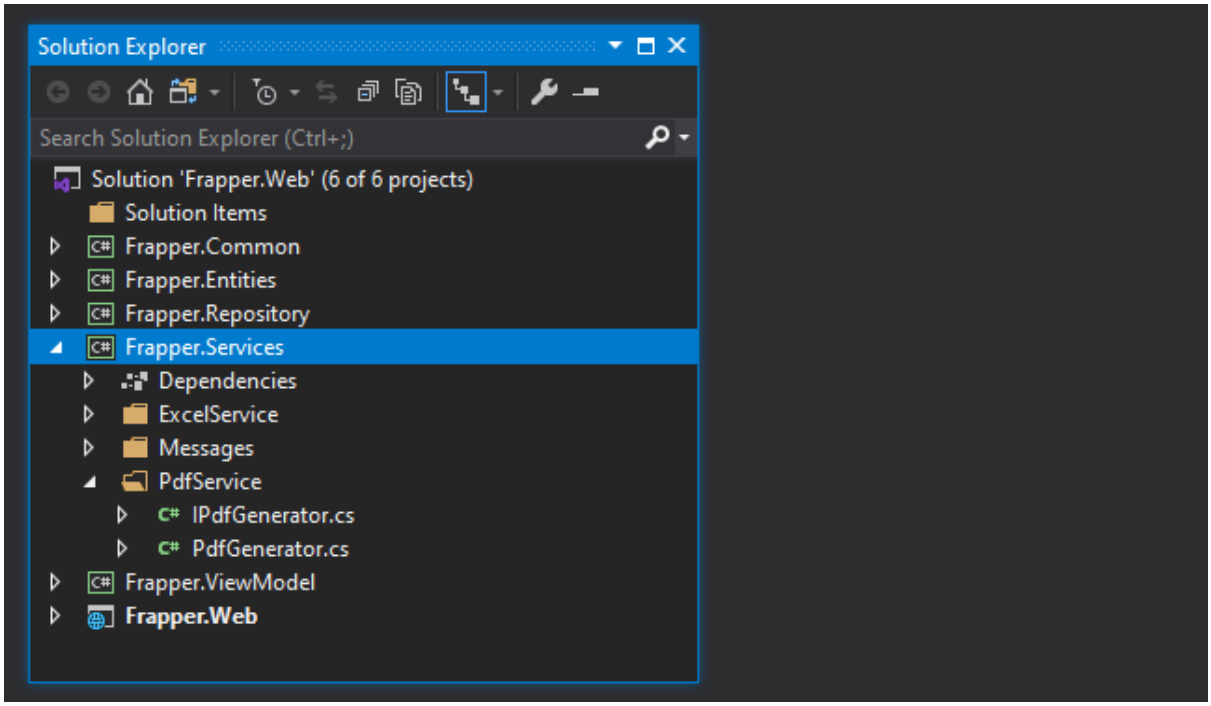
**Command Folder contains all Commands such as Insert, Update, Delete using both Entity Framework and Dapper.**

**Queries Folder contains all Select Queries Using both Entity Framework and Dapper.**



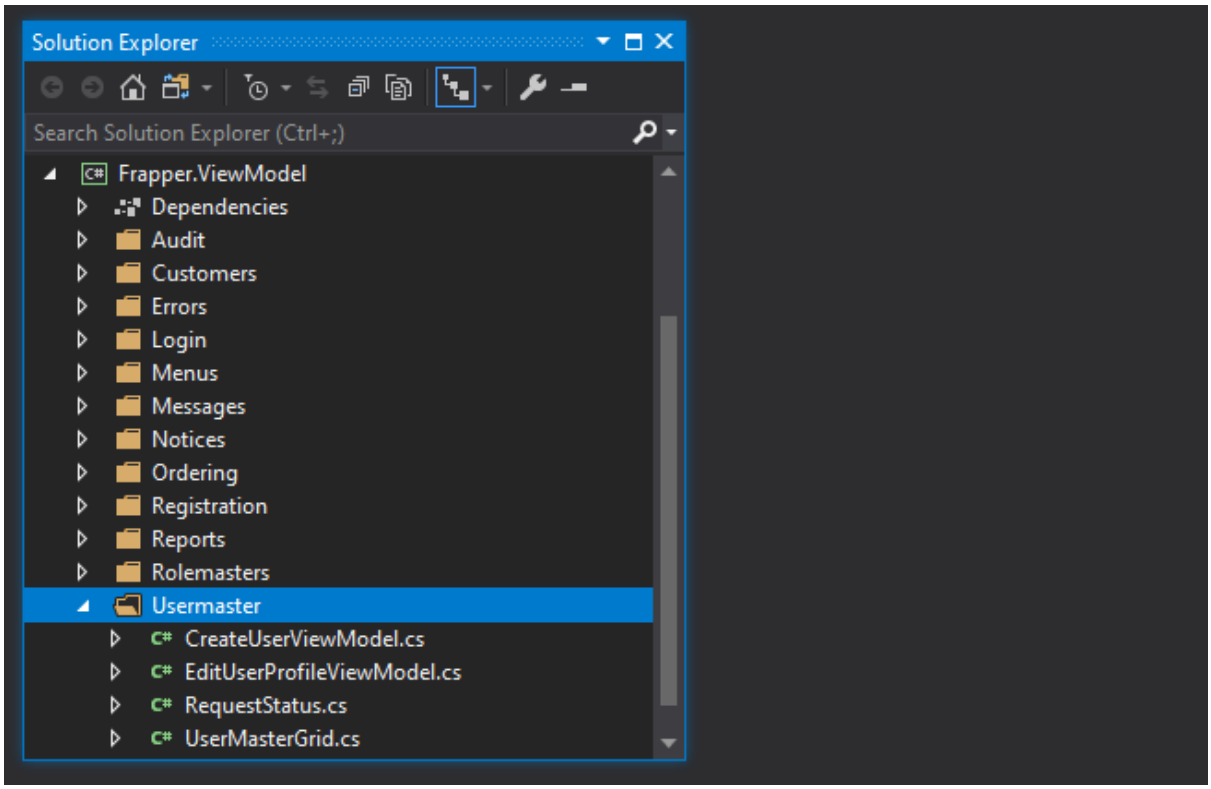

**Frapper.Services**

**Frapper.Services** this project is used to write business logic and also common services such as generating Excel, PDF, Sending Emails. Frapper.Services project has reference to with Frapper.Repository project which means it can access the database directly and Frapper.Services project can be accessed in Frapper.Web project.



**Frapper.ViewModel**

Frapper.ViewModel this project contains all ViewModel used in project.



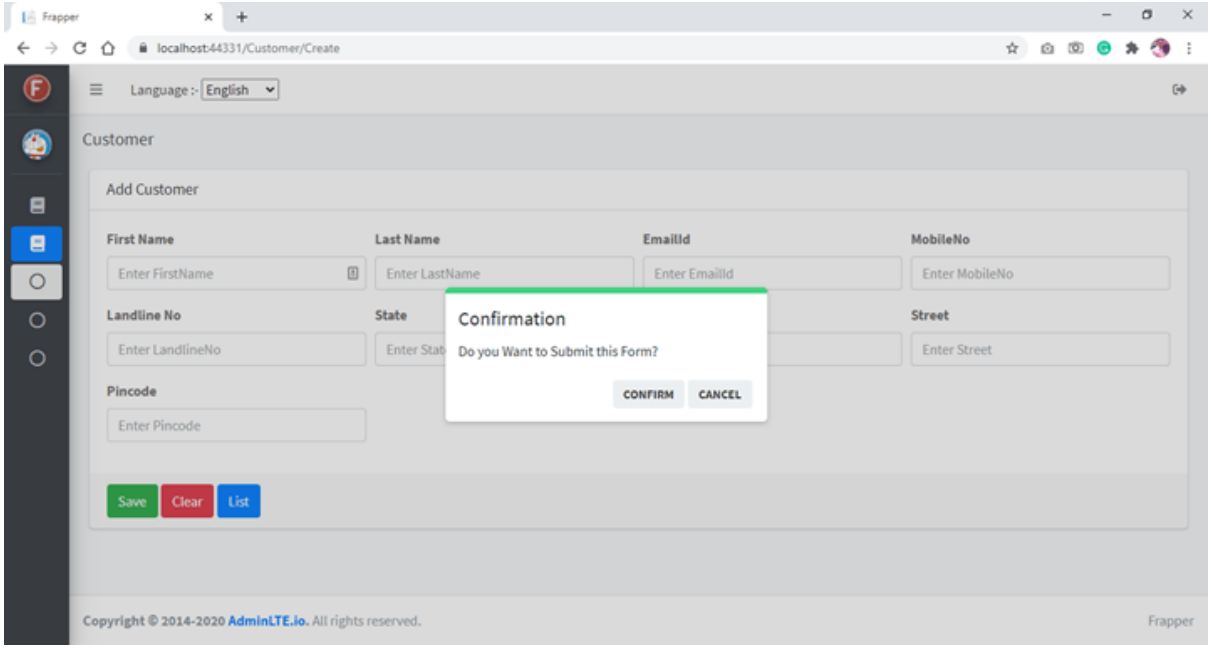

Till now we have seen Project structure if Frapper project now let's Move Forwards and see all features.

---

**Features**

**Alerts**

In this project we have used jQuery Confirm Alerts.

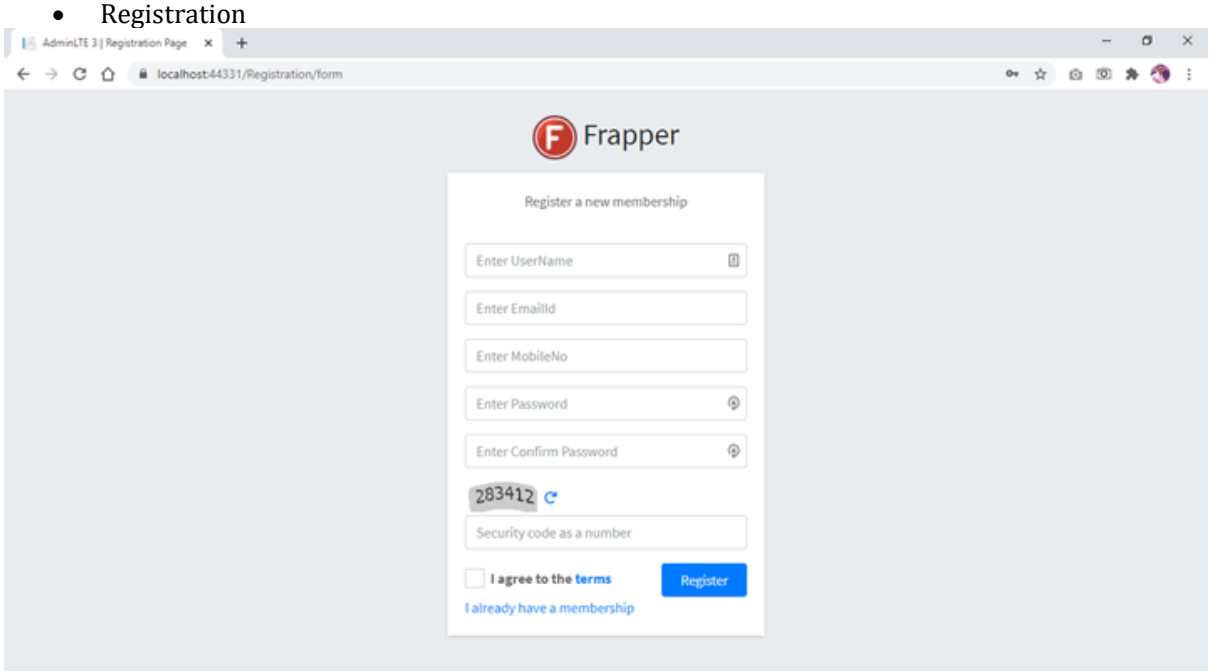


**Custom User Management**

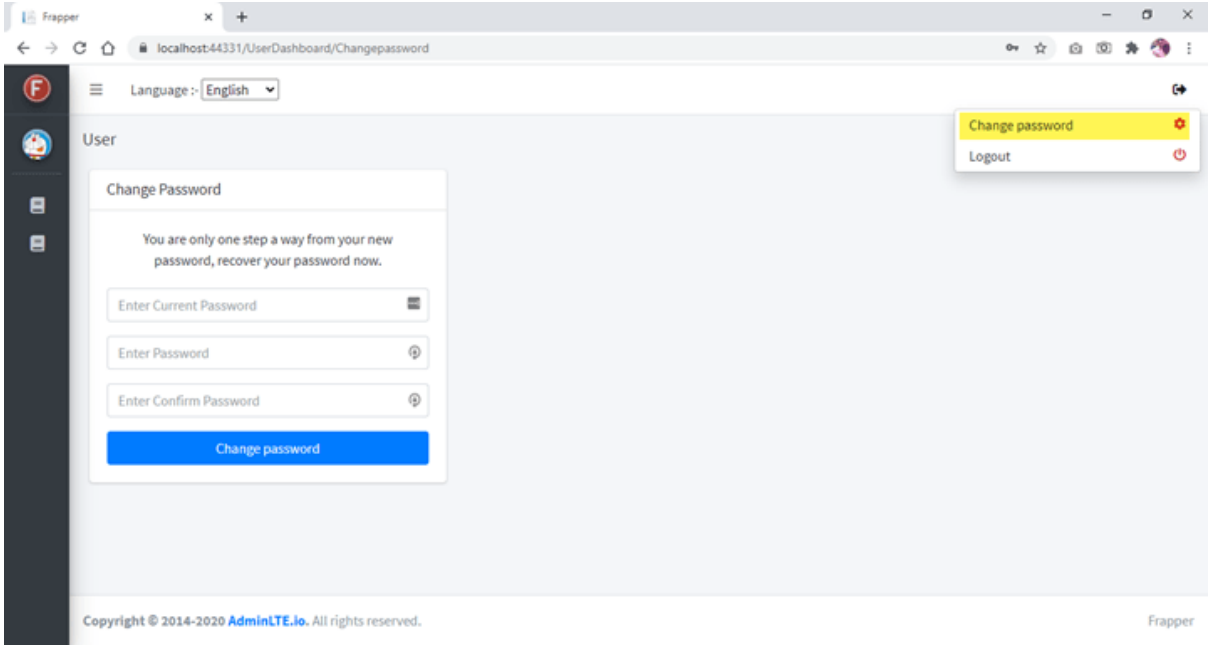
**Custom User Management which contains Login, Registration, Forgot Password, Change Password. Disabling and enabling users.**

All CRUD operations are done using the Repository Pattern and Unit of Work Pattern. Also, we have separated Responsibility into two different part reading (queries) and writing (commands).

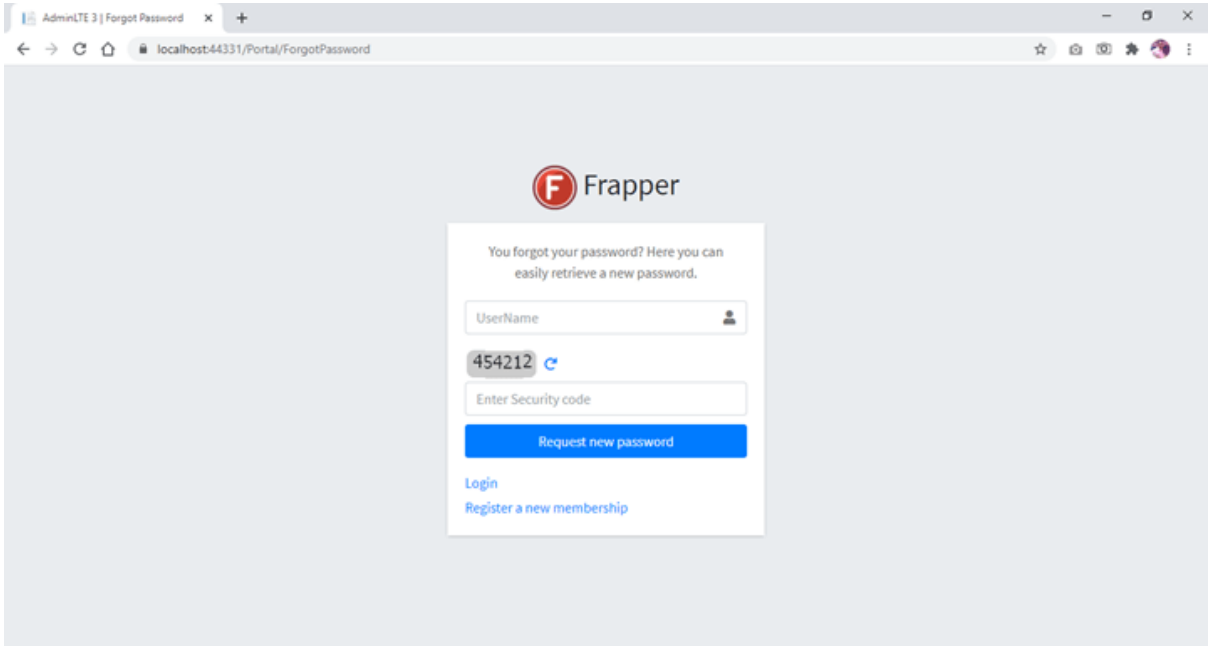
- Registration



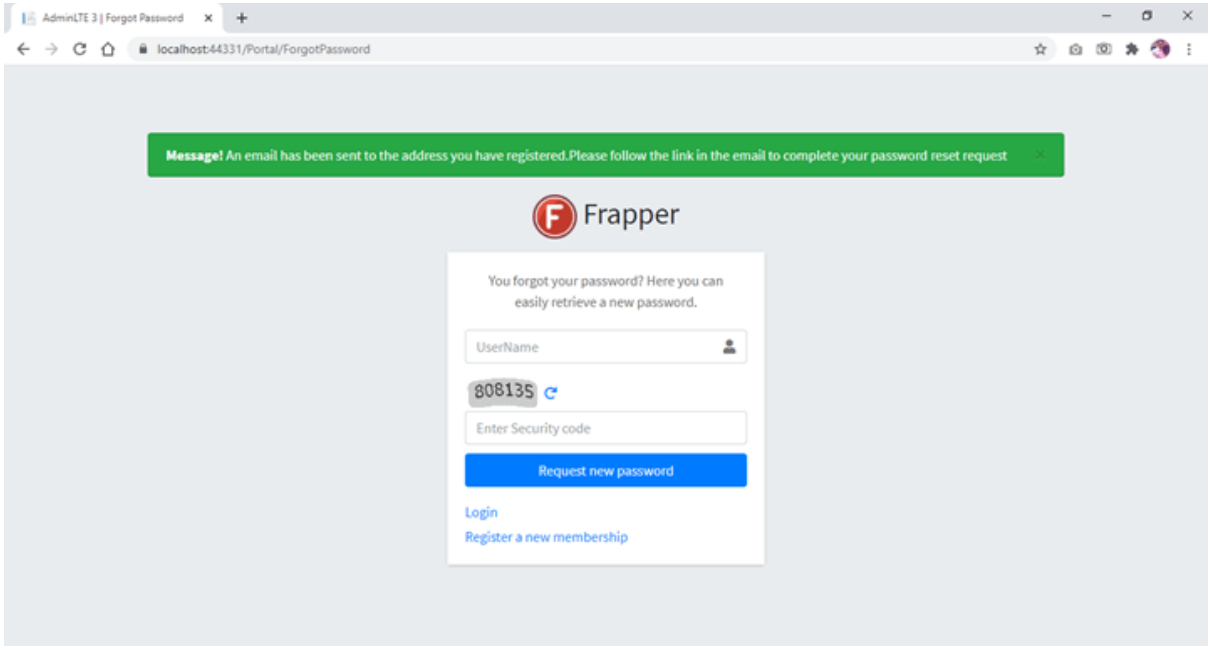

- Change password



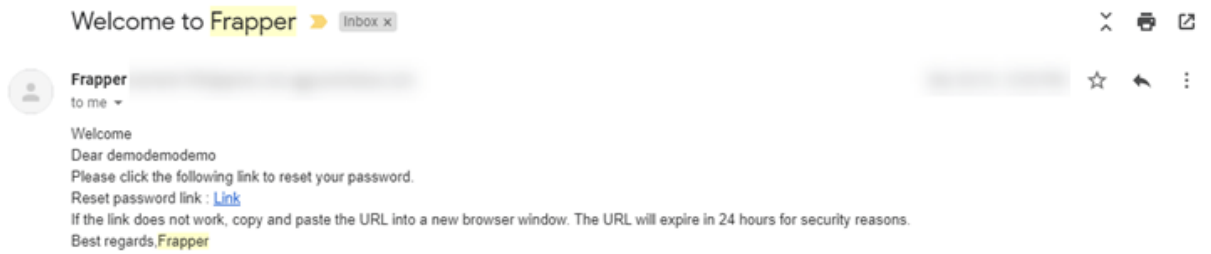
- Forgot Password



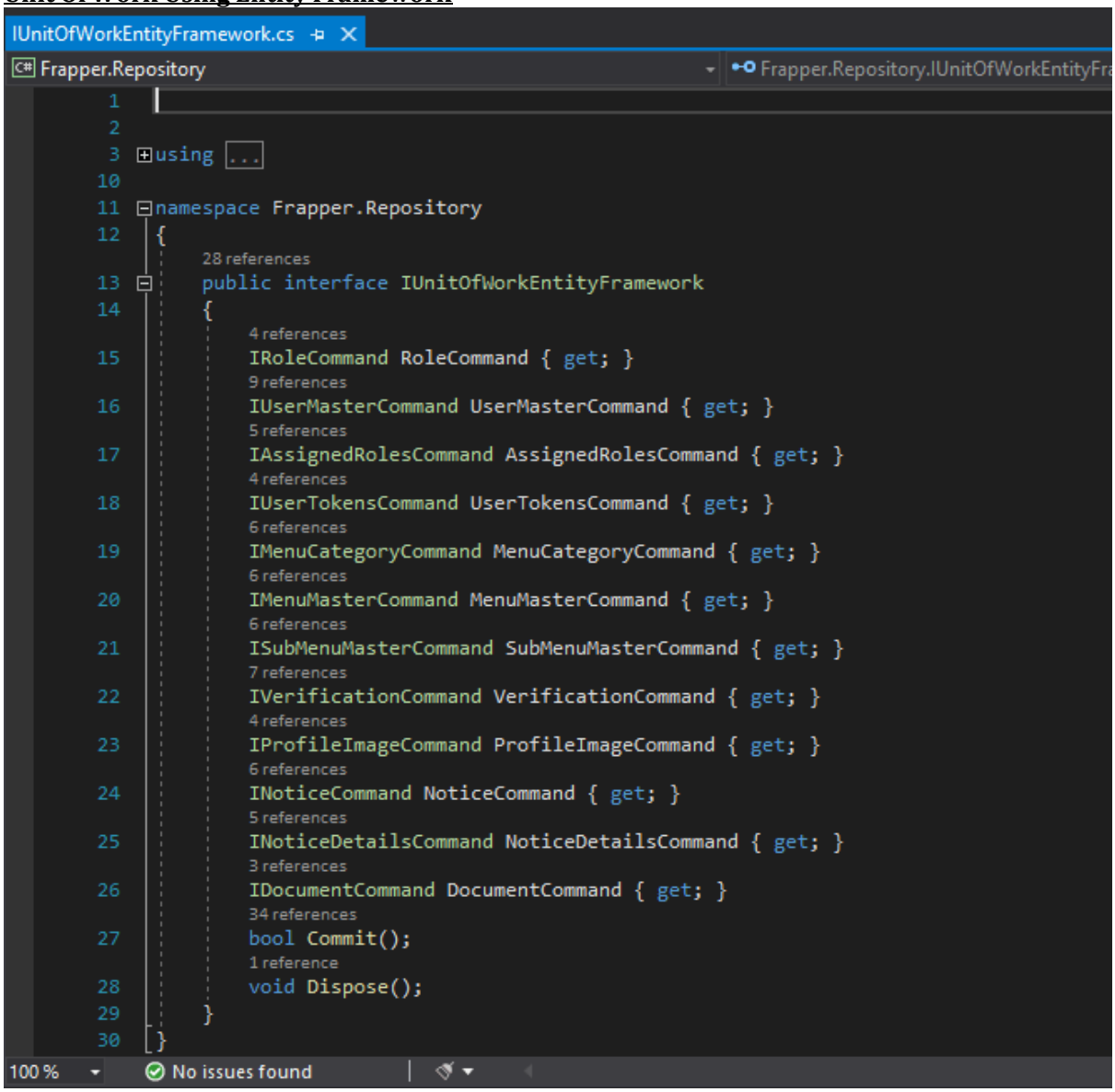

- Email Sent with Reset Password Link



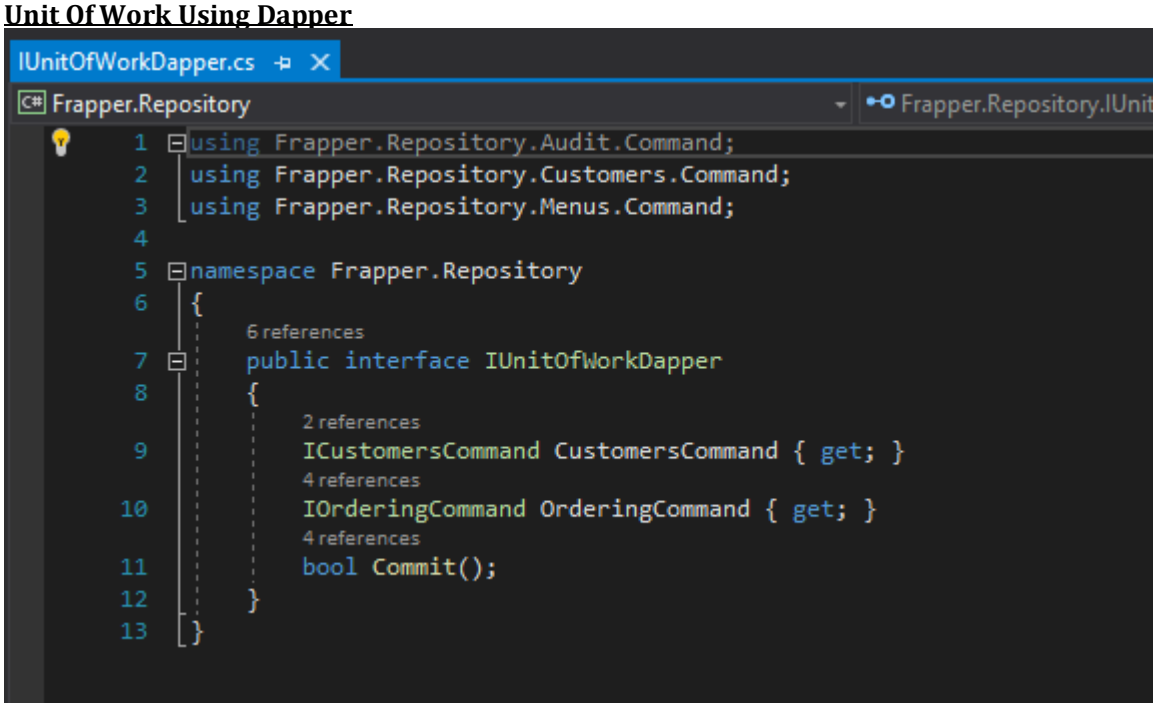

**Email Reset Link**



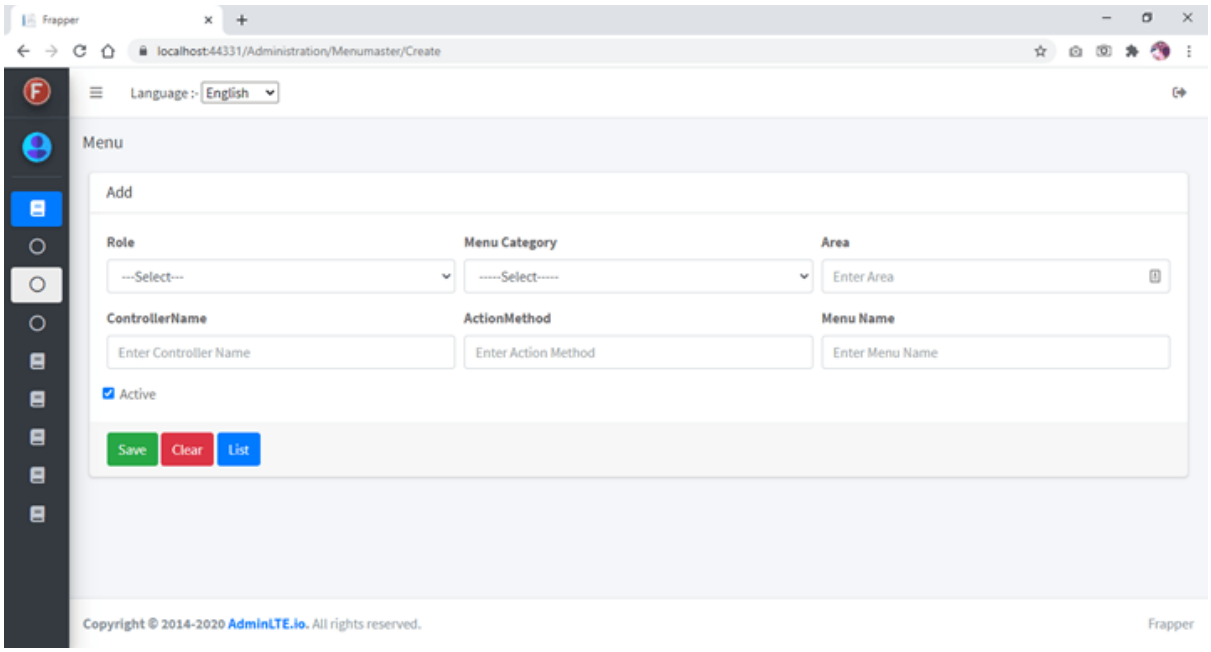
**Unit Of Work Using Entity Framework**



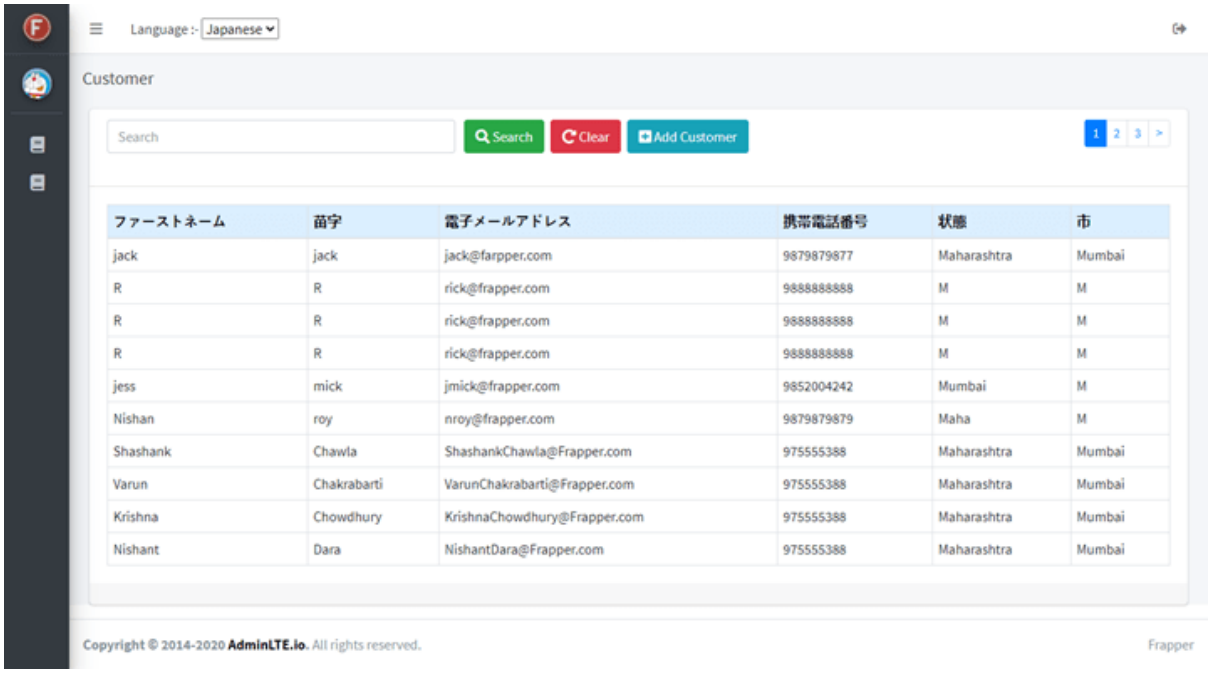

**Unit Of Work Using Dapper**



## Configure Menus by Roles

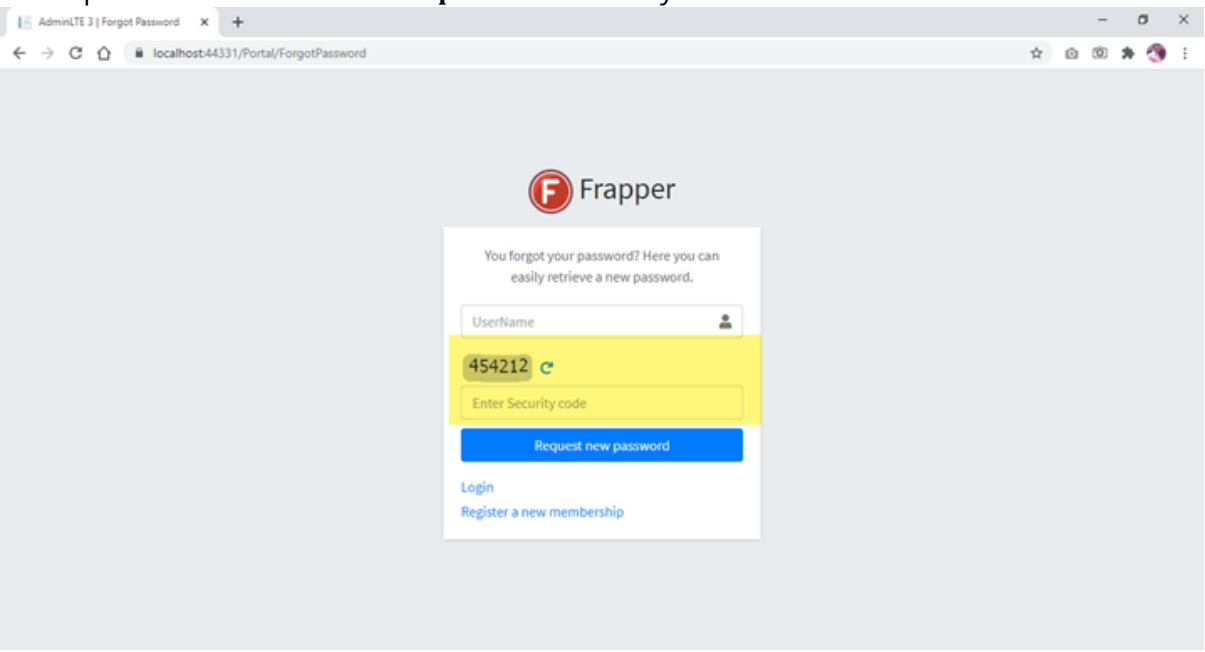



## Custom Grids



## Captcha

For Captcha we have used **DNTCaptcha.Core** Library.

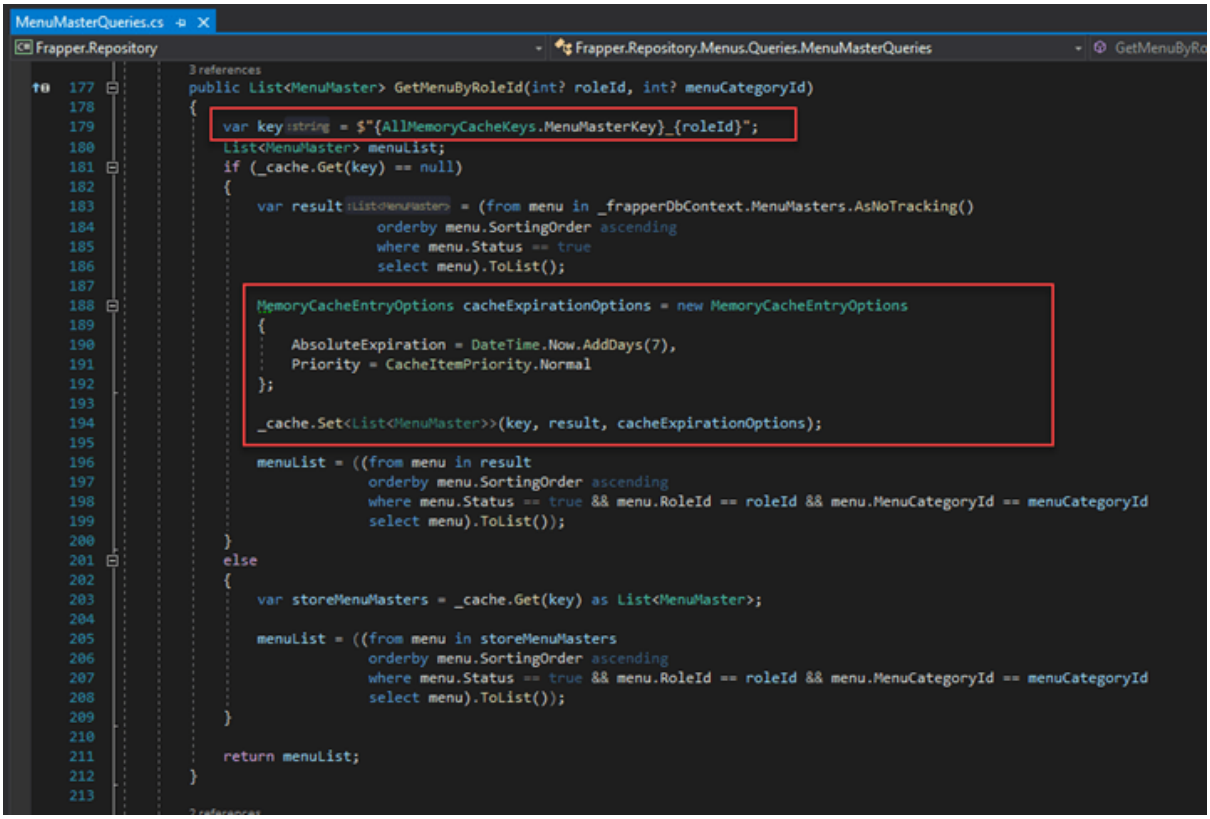



## Cache Services

In this project we are using 2 caching services

1. Memory Cache
2. Redis

## Memory Cache Implementation



## Redis Cache Implementation

Frapper.Web - appsettings.json

appsettings.json

Schema: https://json.schemastore.org/appsettings

```
  }
},
"AllowedHosts": "*",
"ConnectionStrings": ...,
"RedisServer": {
  "Host": "demouow.redis.cache.windows.net",
  "Port": "6380",
  "Password": "HuP9eTqIfCt6mx218Hanlc61PwJFLI1+DohxMqUkMq0=",
  "ssl": "True"
},
"ApplicationSettings": ...,
"Exceptional": ...,
"EPPlus": ...,
"HealthChecksUI": ...
}
```

100 % No issues found Ln: 2 Ch: 15 SPC CRLF

RedisCacheService.cs

Frapper.Repository | Frapper.Repository.CacheService.RedisCacheServi | RedisCacheService(IDistributedCache distributedC

```
using ...

namespace Frapper.Repository.CacheService
{
    2 references
    public class RedisCacheService : RedisCacheServiceBase, IRedisCacheService
    {
        0 references
        public RedisCacheService(IDistributedCache distributedCache) ...

        1 reference
        public void SetObject(string key, object objecttostore, DistributedCacheEntryOptions options)...

        1 reference
        public object GetObject<T>(string key)...

        1 reference
        public void SetString(string key, string stringvalue, DistributedCacheEntryOptions options)...

        1 reference
        public string GetString(string key)...

    }
}
```

## Dapper

Frapper.Web - CustomersCommand.cs

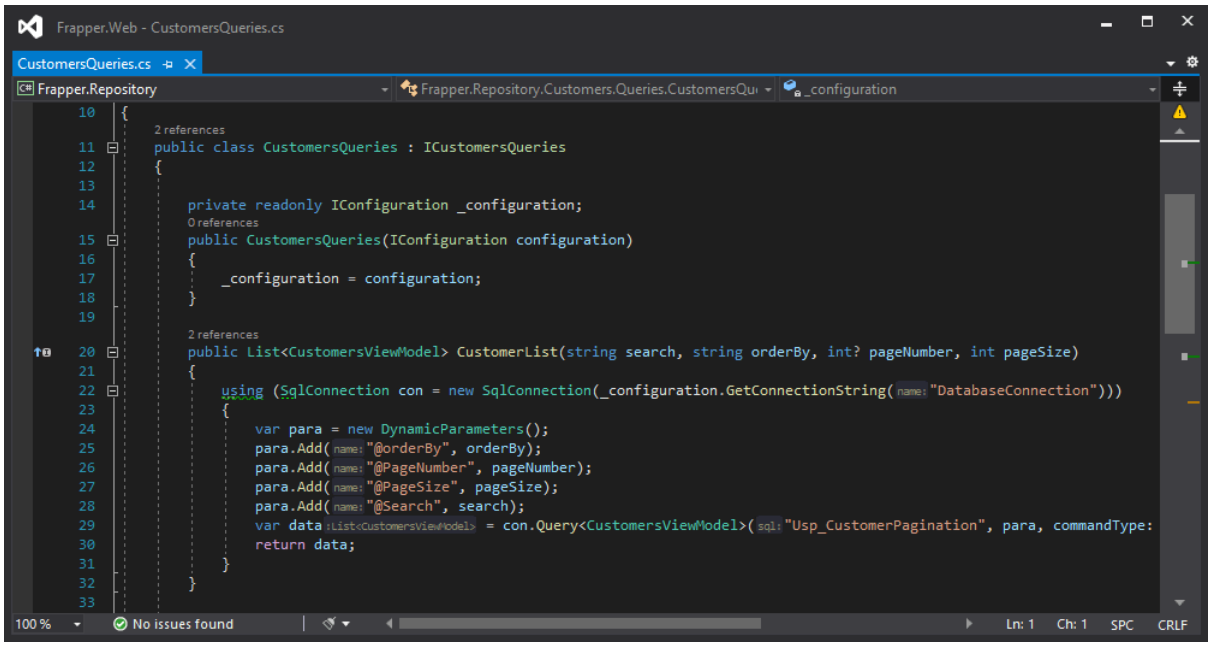
CustomersCommand.cs

Frapper.Repository | Frapper.Repository.Customers.Command.CustomersCommand | _dbConnection

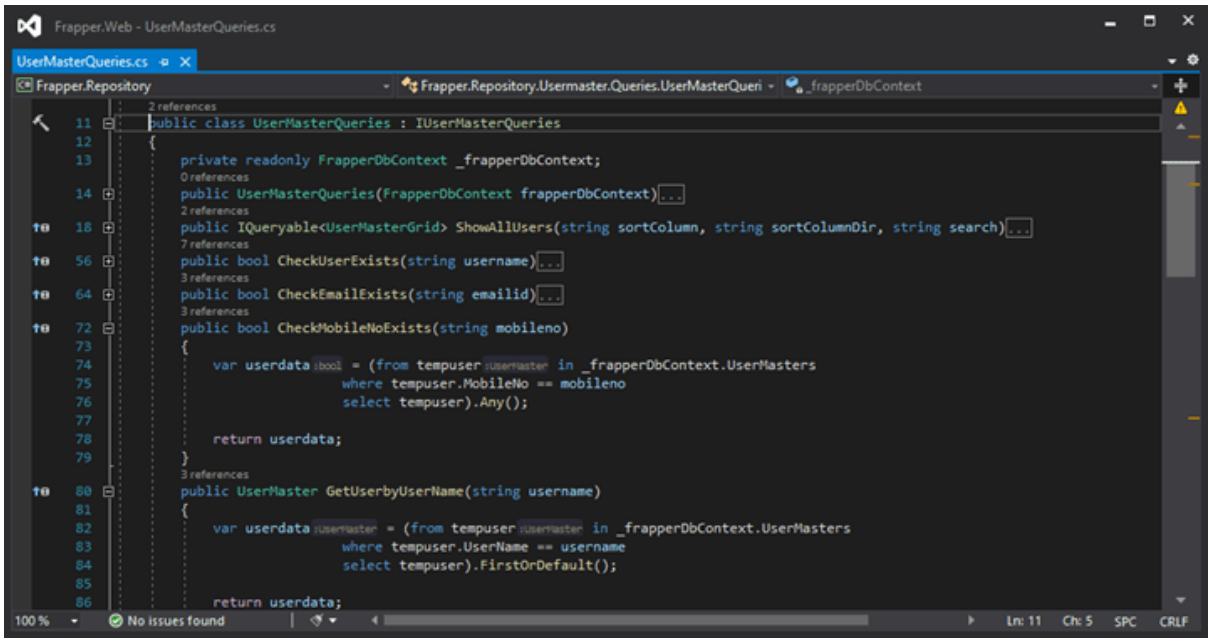
```
using Dapper;
using Frapper.ViewModel.Customers;
using System.Data;

namespace Frapper.Repository.Customers.Command
{
    2 references
    public class CustomersCommand : ICustomersCommand
    {
        private readonly IDbConnection _dbConnection;
        private readonly IDbTransaction _dbTransaction;
        1 reference
        public CustomersCommand(IDbTransaction dbTransaction, IDbConnection dbConnection)
        {
            _dbConnection = dbConnection;
            _dbTransaction = dbTransaction;
        }

        2 references
        public void Add(CustomersViewModel customersViewModel, long userId)
        {
            var param = new DynamicParameters();
            param.Add(name: "@FirstName", customersViewModel.FirstName);
            param.Add(name: "@LastName", customersViewModel.LastName);
            param.Add(name: "@MobileNo", customersViewModel.MobileNo);
            param.Add(name: "@LandlineNo", customersViewModel.LandlineNo);
            param.Add(name: "@EmailId", customersViewModel.EmailId);
            param.Add(name: "@Street", customersViewModel.Street);
            param.Add(name: "@City", customersViewModel.City);
            param.Add(name: "@State", customersViewModel.State);
            param.Add(name: "@Pincode", customersViewModel.Pincode);
            param.Add(name: "@CreatedBy", userId);
            _dbConnection.Execute(sql: "Usp_InsertCustomer", param, _dbTransaction, commandTimeout: 0, CommandType.StoredProcedure);
```

100 % No issues found

## Dapper

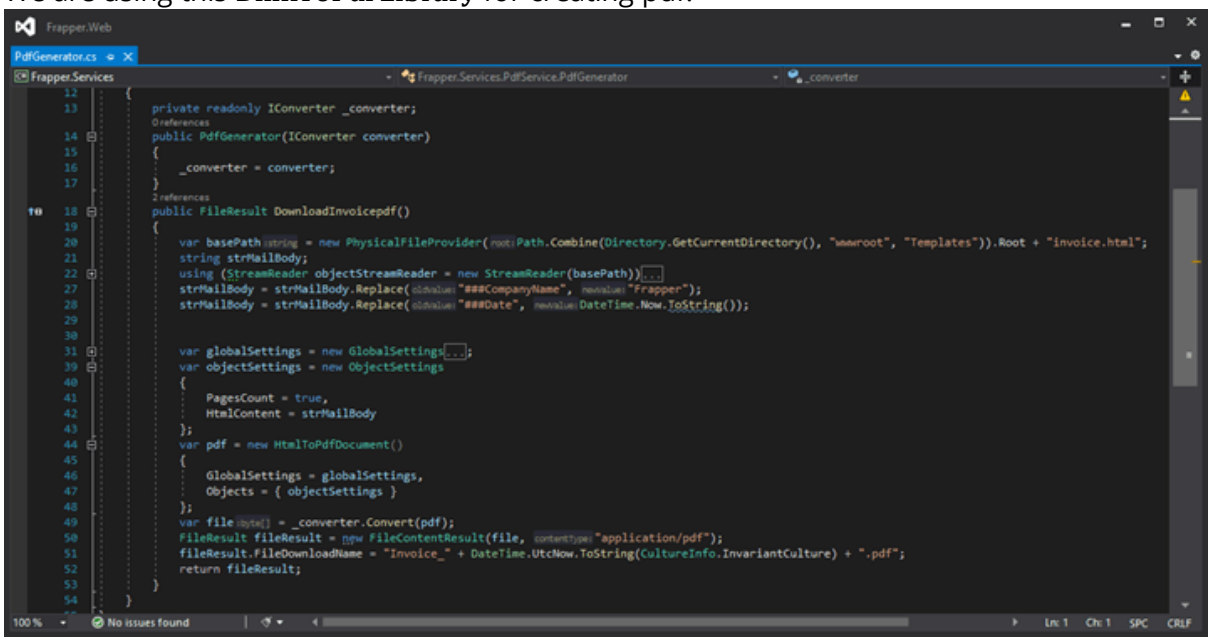



## Entity Framework Core

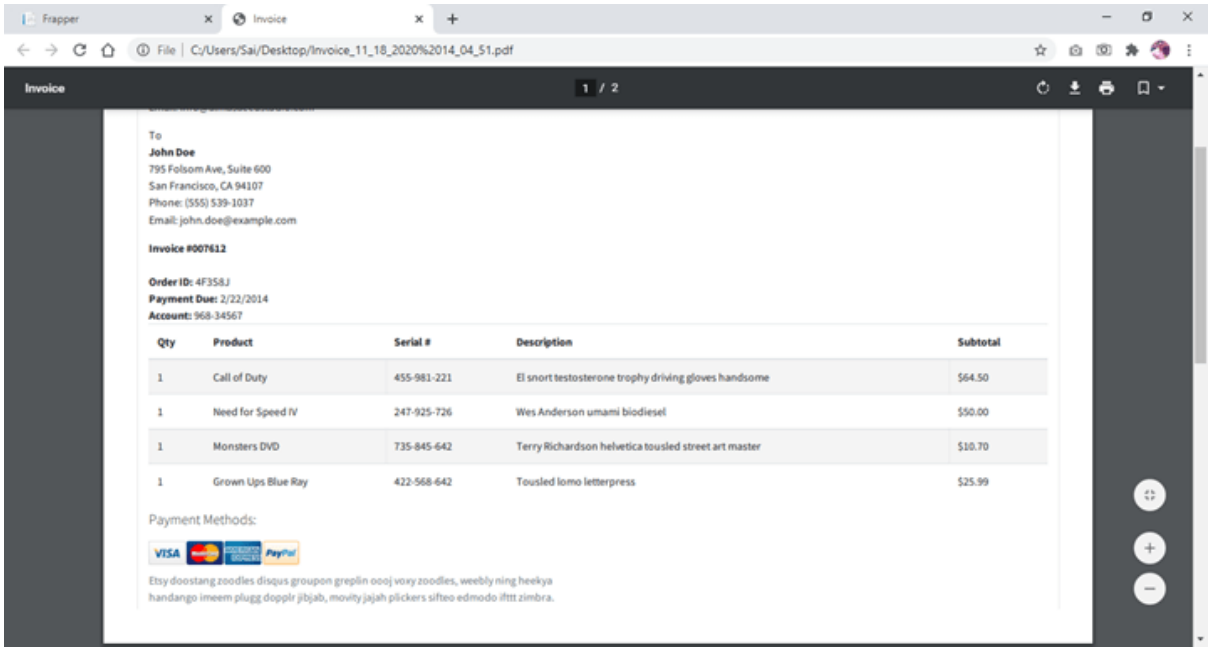


## Create Pdf files

We are using this **DinkToPdf Library** for creating pdf.



```
{
    private readonly IConverter _converter;
    0 references
    public PdfGenerator(IConverter converter)
    {
        _converter = converter;
    }
    2 references
    public FileResult DownloadInvoicepdf()
    {
        var basePath = new PhysicalFileProvider(Path.Combine(Directory.GetCurrentDirectory(), "wwwroot", "Templates")).Root + "invoice.html";
        string strMailBody;
        using (StreamReader objectStreamReader = new StreamReader(basePath))
        strMailBody = strMailBody.Replace("###CompanyName", "Frapper");
        strMailBody = strMailBody.Replace("###Date", DateTime.Now.ToString());


        var globalSettings = new GlobalSettings...;
        var objectSettings = new ObjectSettings
        {
            PagesCount = true,
            HtmlContent = strMailBody
        };
        var pdf = new HtmlToPdfDocument()
        {
            GlobalSettings = globalSettings,
            Objects = { objectSettings }
        };
        var file = _converter.Convert(pdf);
        FileResult fileResult = new FileContentResult(file, "application/pdf");
        fileResult.FileDownloadName = "Invoice_" + DateTime.UtcNow.ToString(CultureInfo.InvariantCulture) + ".pdf";
        return fileResult;
    }
}
```



Invoice

To
**John Doe**
795 Folsom Ave, Suite 600
San Francisco, CA 94107
Phone: (555) 539-1037
Email: john.doe@example.com

**Invoice #007612**

**Order ID:** 4F3S8J
**Payment Due:** 2/22/2014
**Account:** 968-34567

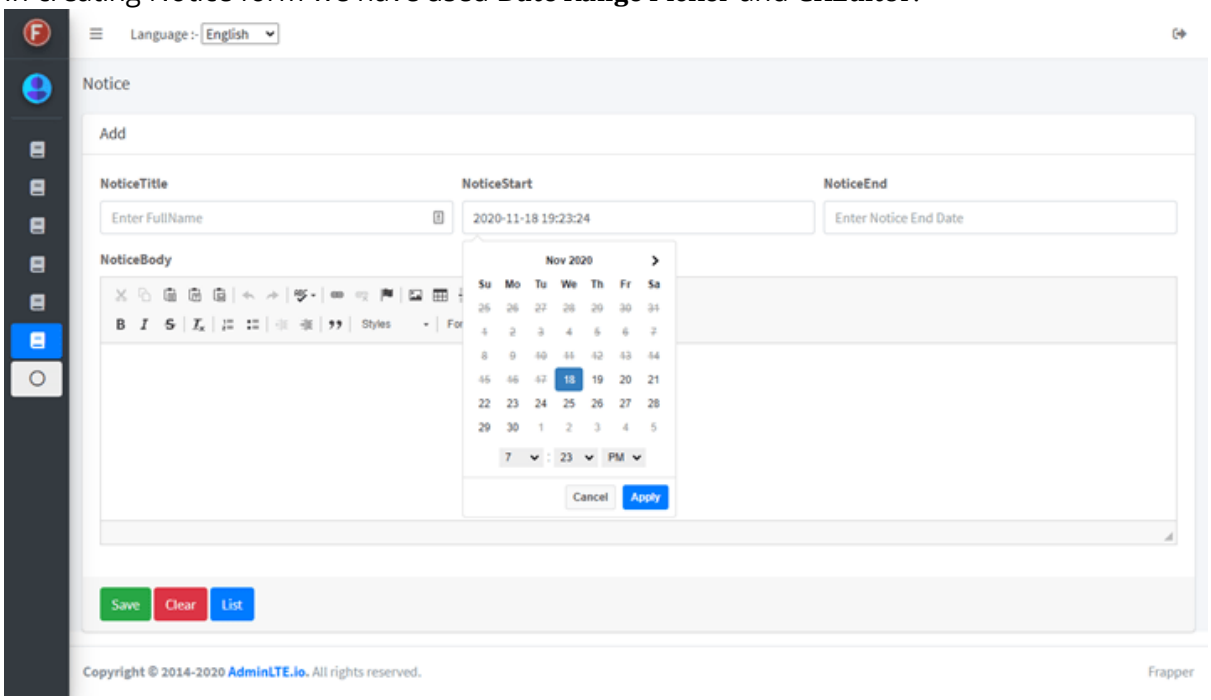
| Qty | Product | Serial # | Description | Subtotal |
|---|---|---|---|---|
| 1 | Call of Duty | 455-981-221 | El snort testosterone trophy driving gloves handsome | $64.50 |
| 1 | Need for Speed IV | 247-925-726 | Wes Anderson umami biodiesel | $50.00 |
| 1 | Monsters DVD | 735-845-642 | Terry Richardson helvetica tousled street art master | $10.70 |
| 1 | Grown Ups Blue Ray | 422-568-642 | Tousled lomo letterpress | $25.99 |

Payment Methods:

Etsy doostang zoodles disqus groupon greplin oooj voxy zoodles, weebly ning heekya handango imeem plugg dopplr jibjab, movity jajah plickers sifteo edmodo ifttt zimbra.

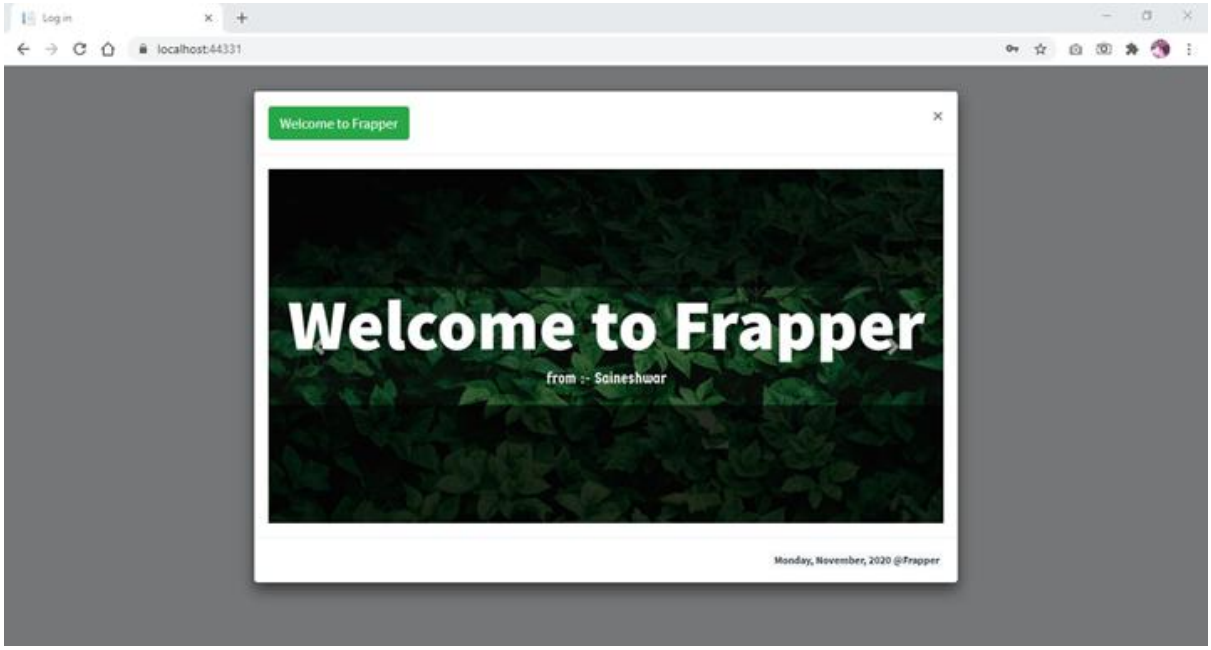
## Create Notice

In creating Notice form we have used **Date Range Picker** and **CKEditor**.

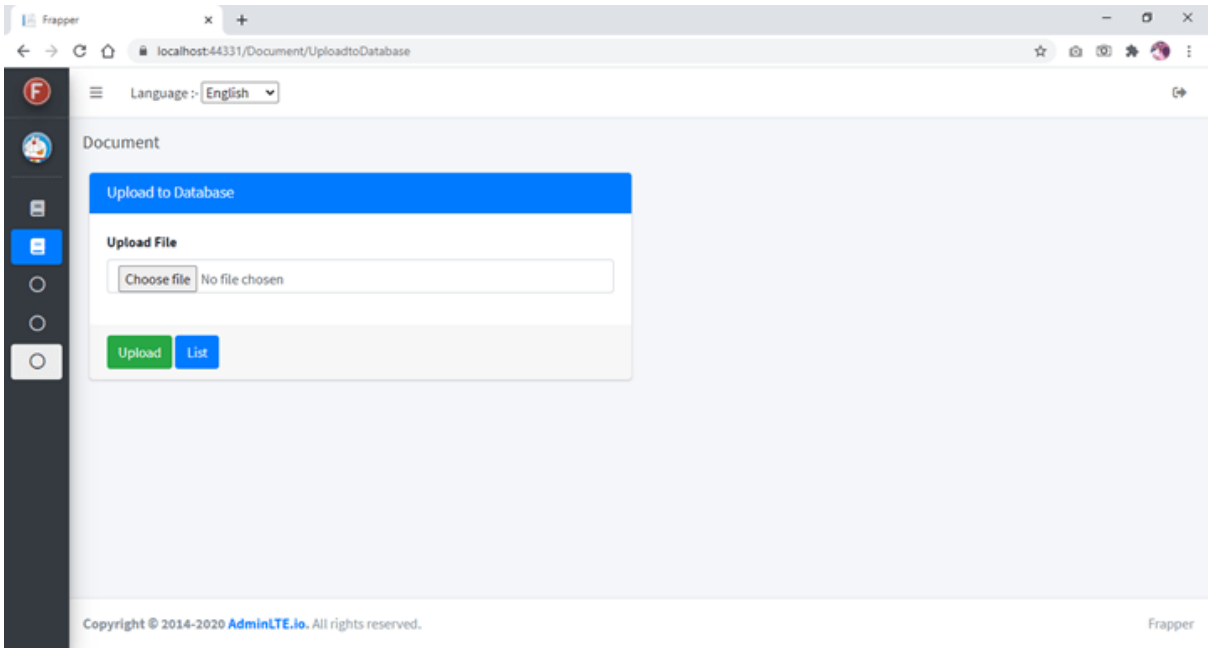


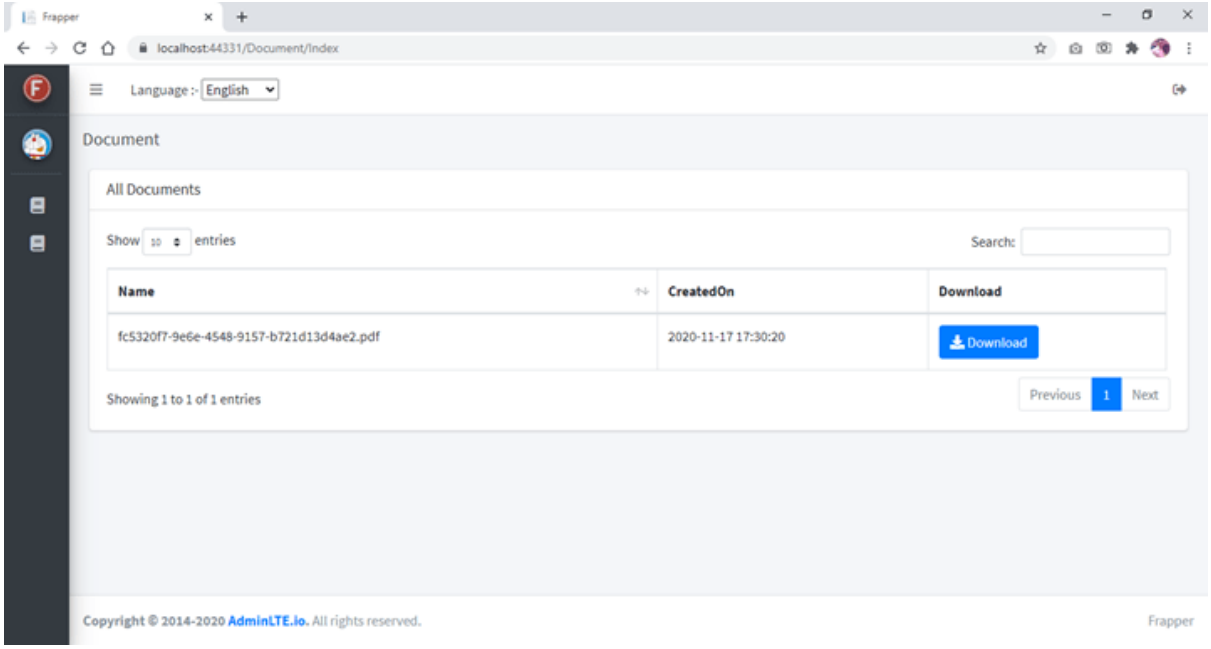

## Rendering Notice

For rendering Notice we have used Bootstrap Modal pop.



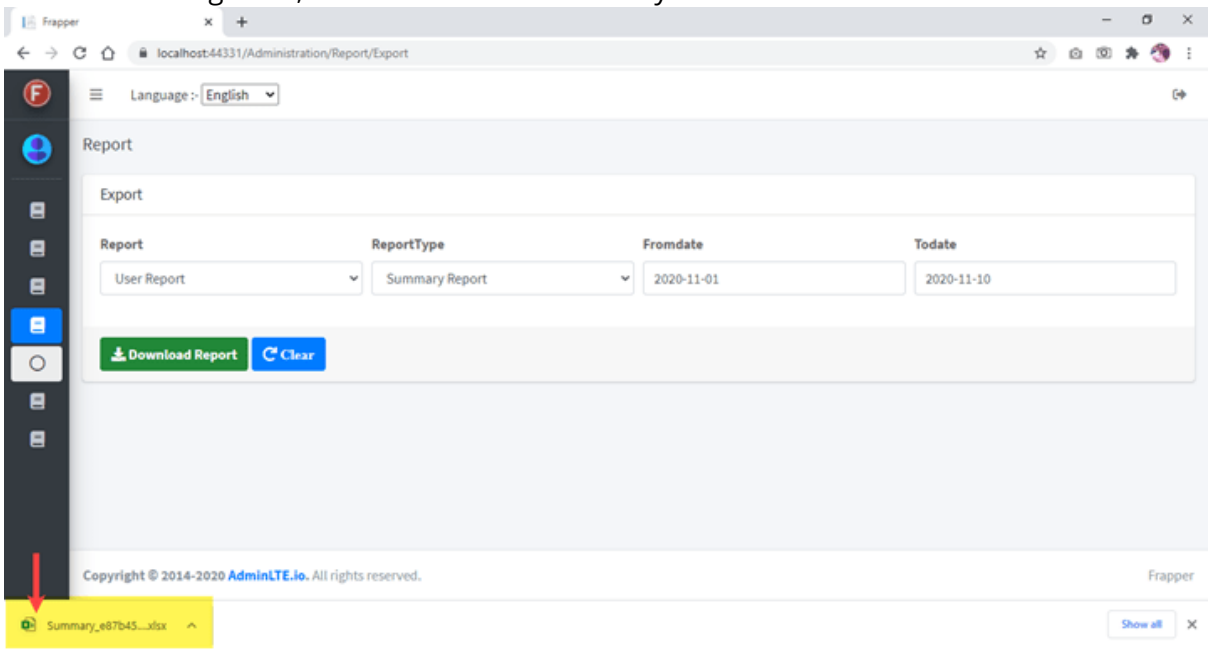
## Download Files



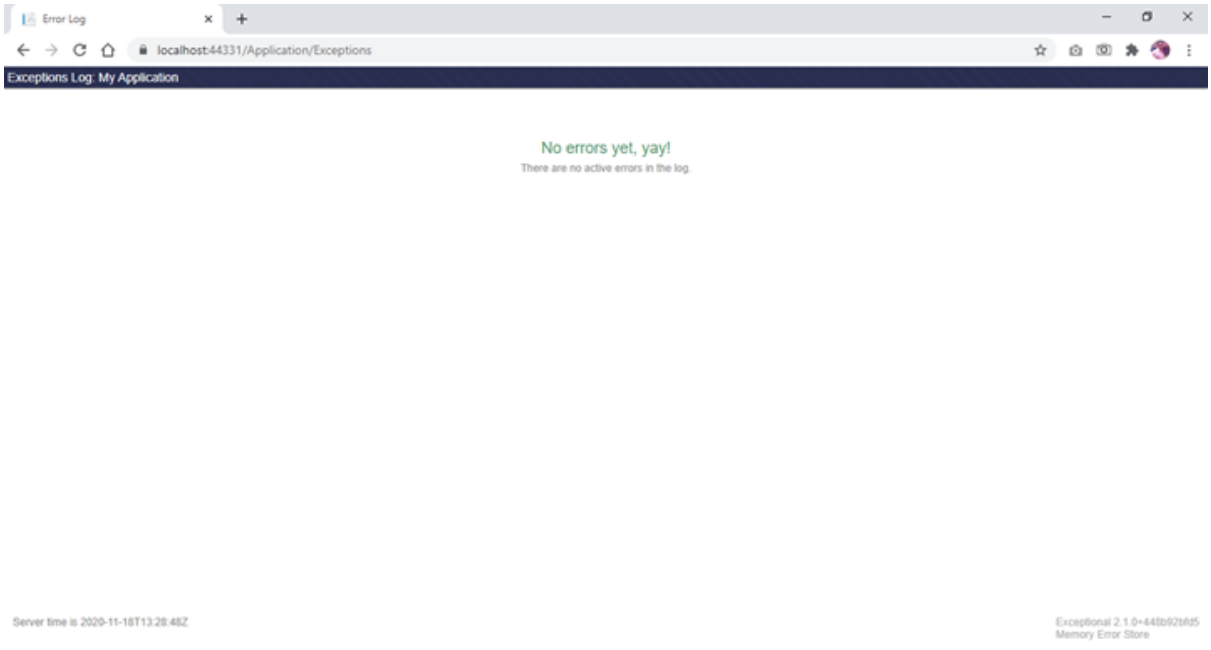
## Download Excel Reports

For downloading Excel, we have used **EPLUS** library.



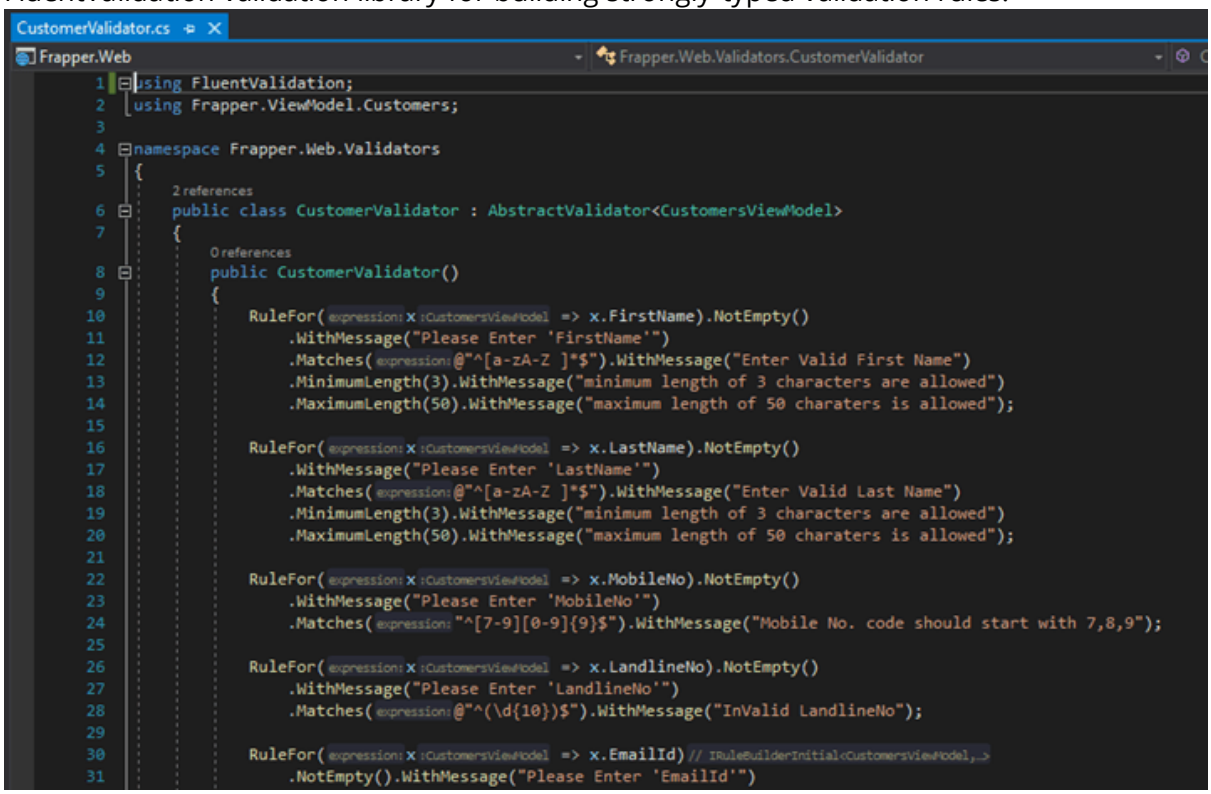

## Exception Logging

For Logging exception, we have to use StackExchange.Exceptional.AspNetCore.



## FluentValidation

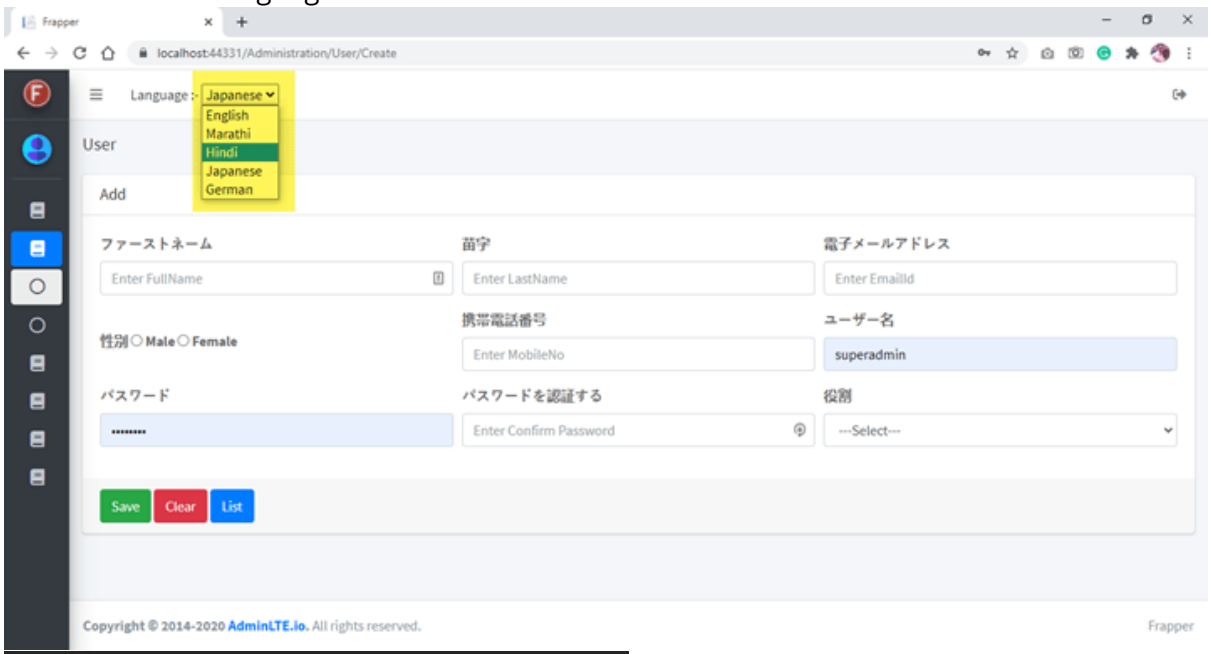
FluentValidation validation library for building strongly-typed validation rules.

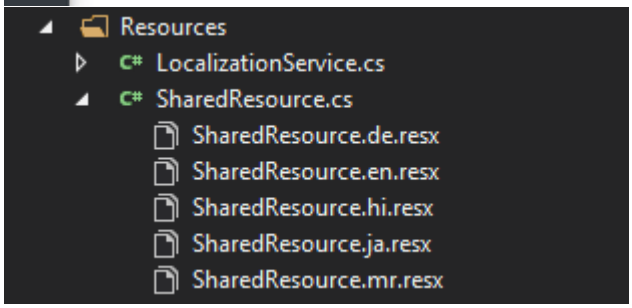


```
CustomerValidator.cs
Frapper.Web
Frapper.Web.Validators.CustomerValidator
1  using FluentValidation;
2  using Frapper.ViewModel.Customers;
3
4  namespace Frapper.Web.Validators
5  {
       2 references
6      public class CustomerValidator : AbstractValidator<CustomersViewModel>
7      {
           0 references
8          public CustomerValidator()
9          {
10             RuleFor(expression: x => x.FirstName).NotEmpty()
11                 .WithMessage("Please Enter 'FirstName'")
12                 .Matches(expression: @"^[a-zA-Z ]*$").WithMessage("Enter Valid First Name")
13                 .MinimumLength(3).WithMessage("minimum length of 3 characters are allowed")
14                 .MaximumLength(50).WithMessage("maximum length of 50 charaters is allowed");
15
16             RuleFor(expression: x => x.LastName).NotEmpty()
17                 .WithMessage("Please Enter 'LastName'")
18                 .Matches(expression: @"^[a-zA-Z ]*$").WithMessage("Enter Valid Last Name")
19                 .MinimumLength(3).WithMessage("minimum length of 3 characters are allowed")
20                 .MaximumLength(50).WithMessage("maximum length of 50 charaters is allowed");
21
22             RuleFor(expression: x => x.MobileNo).NotEmpty()
23                 .WithMessage("Please Enter 'MobileNo'")
24                 .Matches(expression: "^[7-9][0-9]{9}$").WithMessage("Mobile No. code should start with 7,8,9");
25
26             RuleFor(expression: x => x.LandlineNo).NotEmpty()
27                 .WithMessage("Please Enter 'LandlineNo'")
28                 .Matches(expression: @"^(\d{10})$").WithMessage("InValid LandlineNo");
29
30             RuleFor(expression: x => x.EmailId) // IRuleBuilderInitial<CustomersViewModel,...>
31                 .NotEmpty().WithMessage("Please Enter 'EmailId'")
```

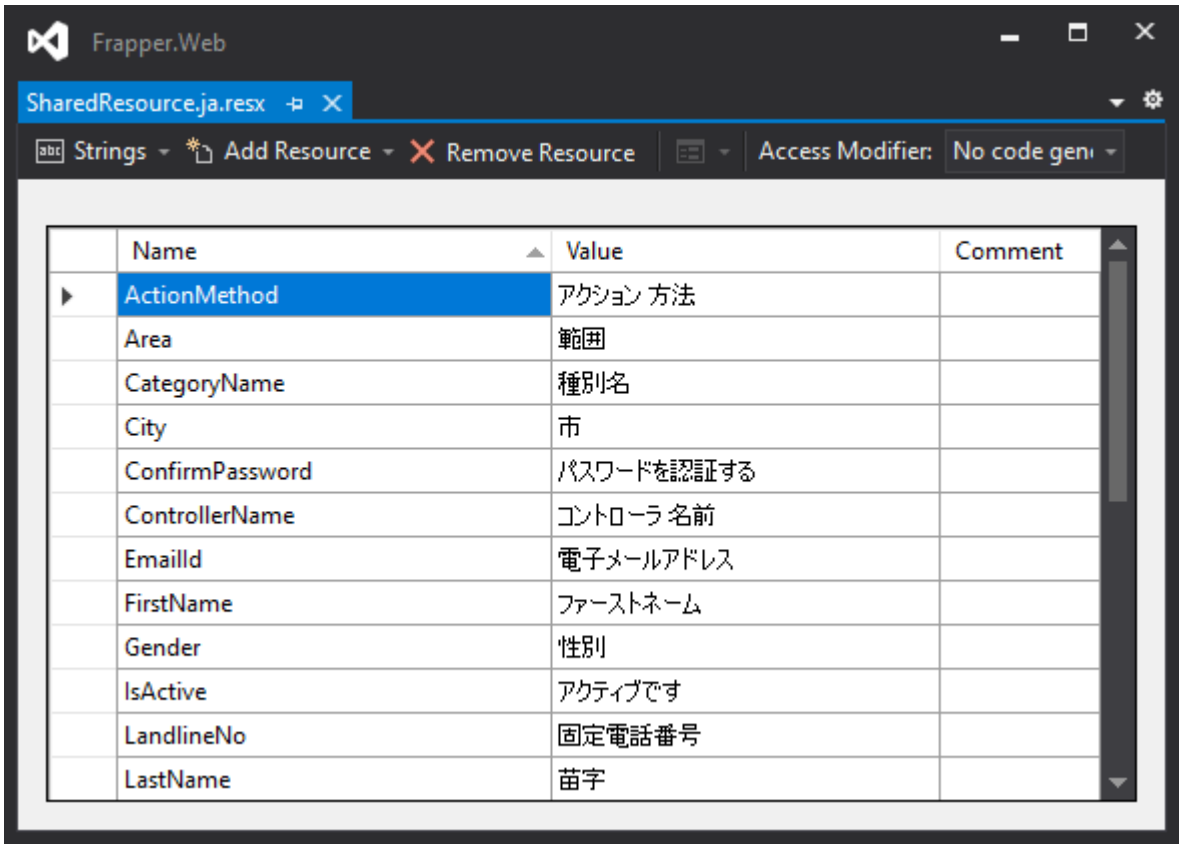
## Globalization and Localization

We have used 5 languages for demo. For that we have created 5 resource files.

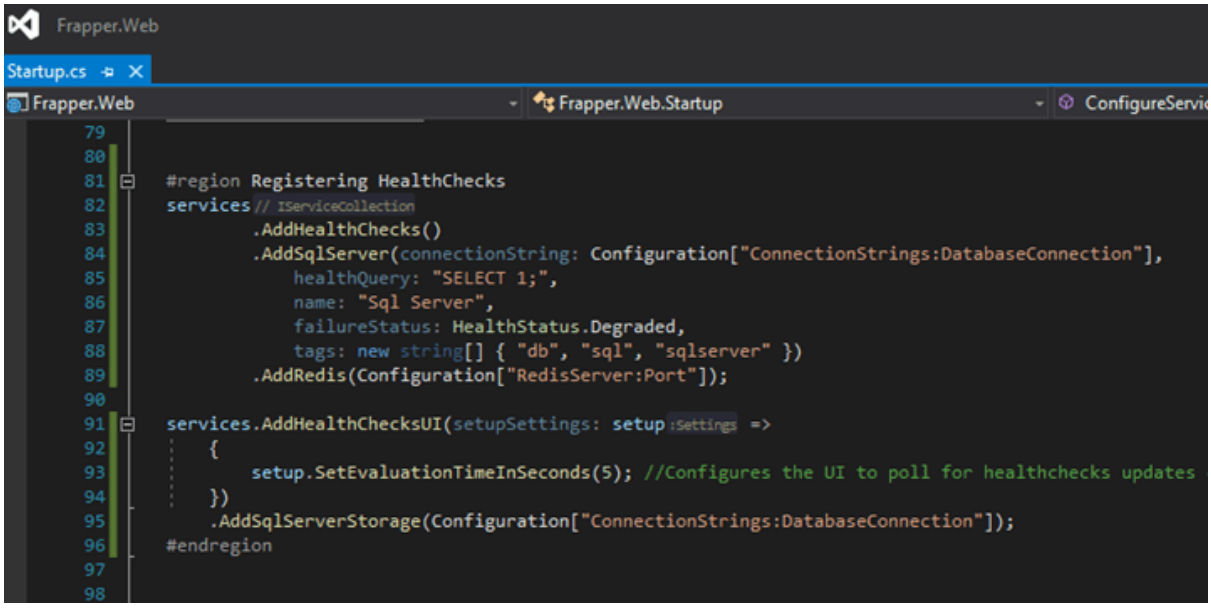


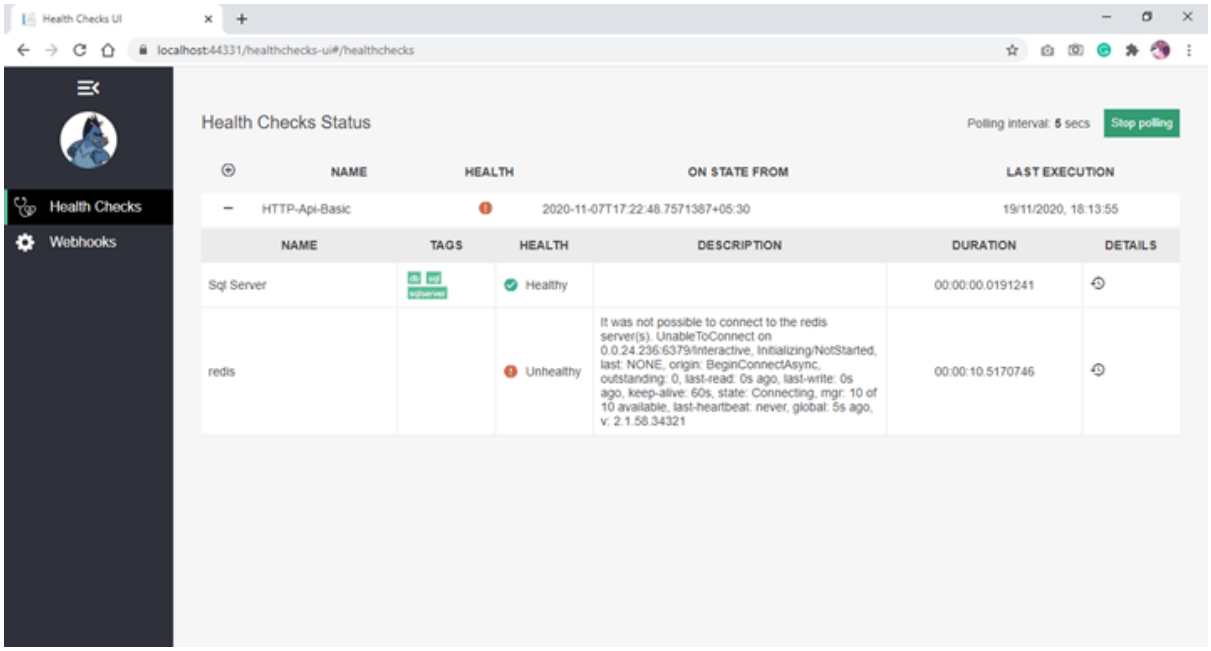
**Resource file**




**HealthChecks**

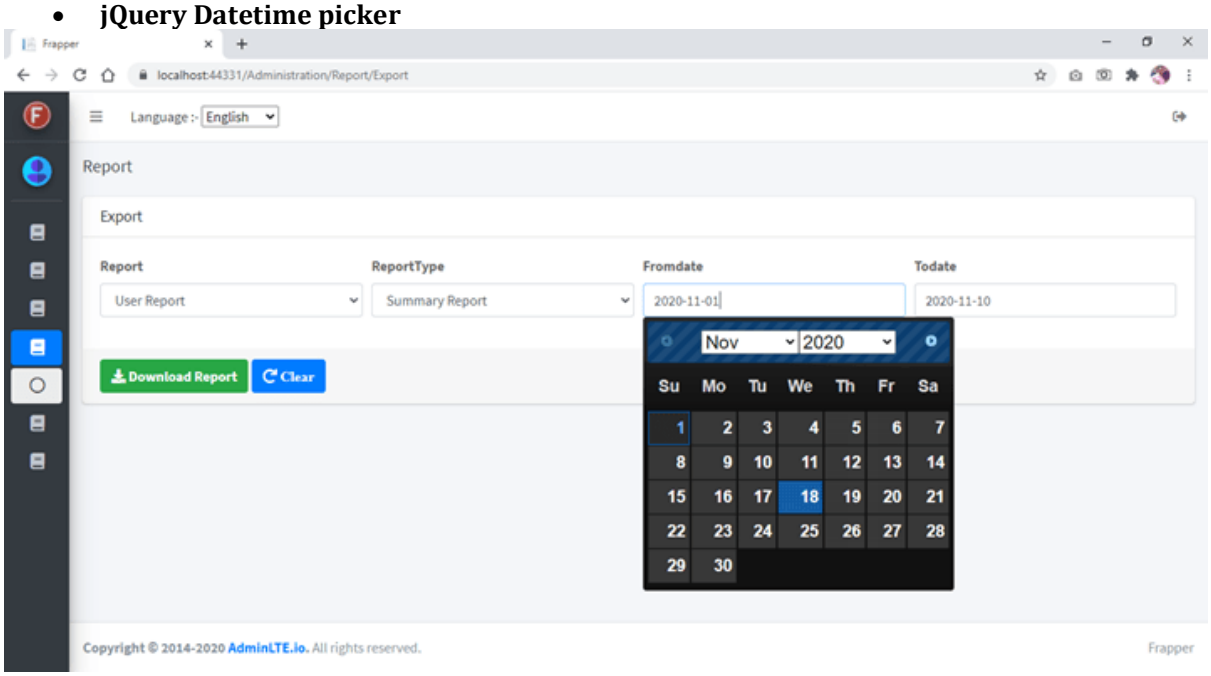


## HealthChecks

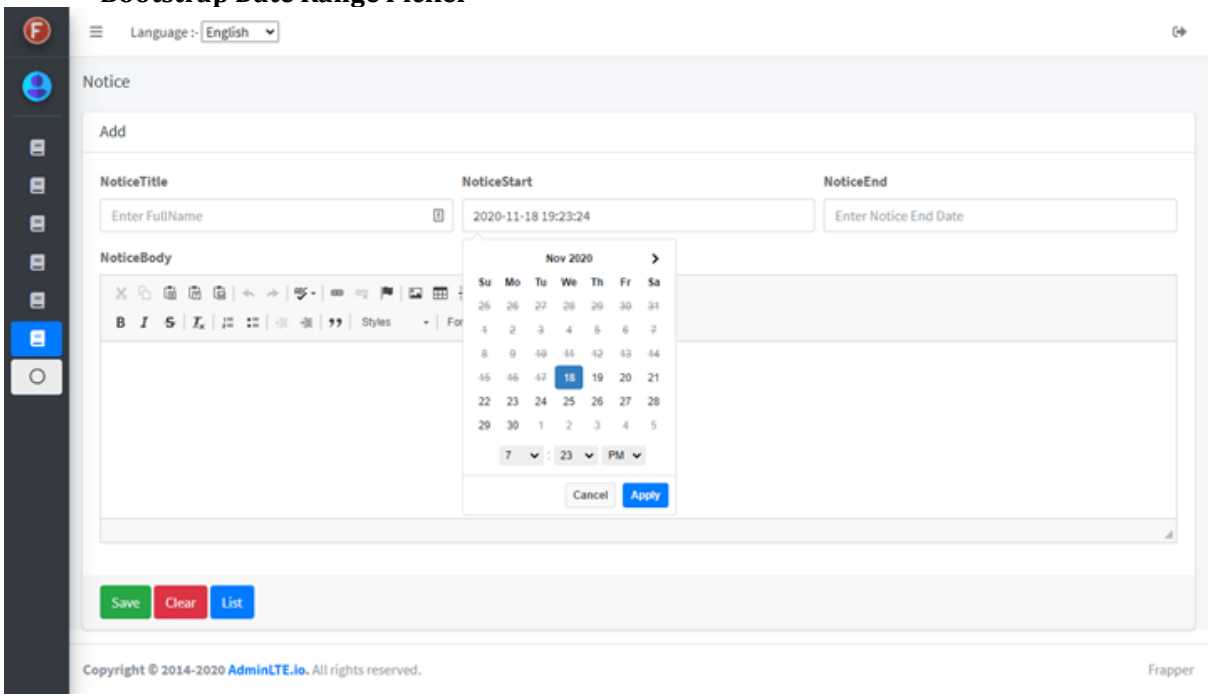



## jQuery Datetime pickers & Bootstrap Datetime pickers

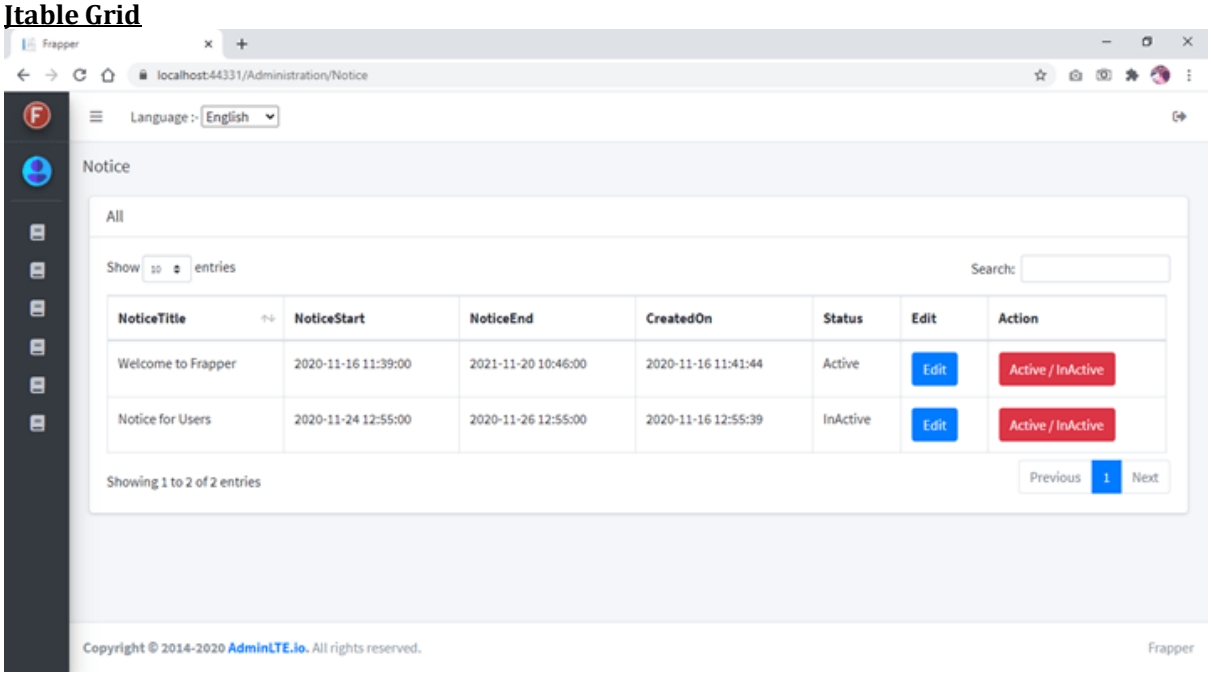
- **jQuery Datetime picker**



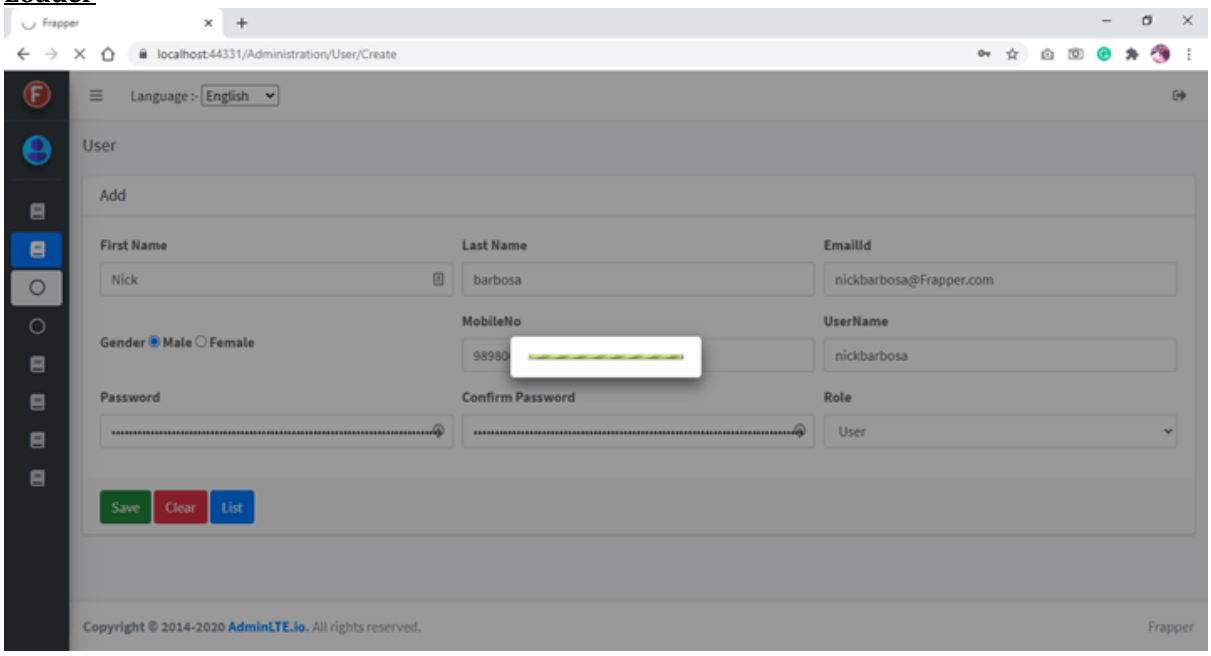
- **Bootstrap Date Range Picker**




## Jtable Grid

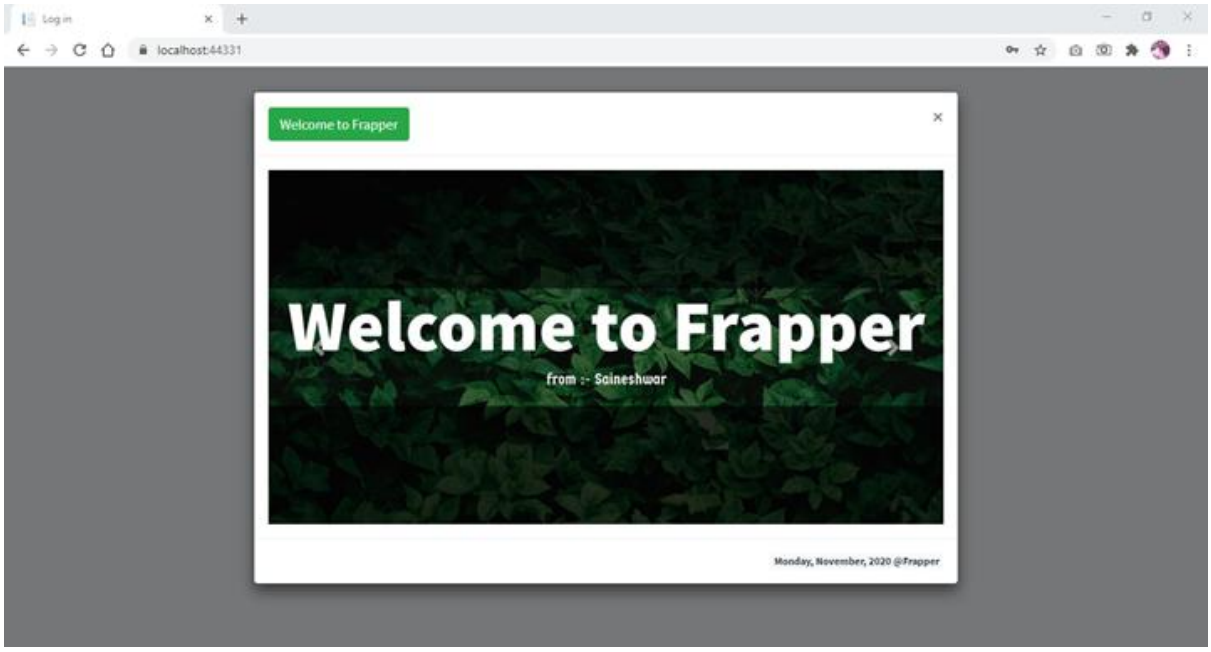


**Loader**

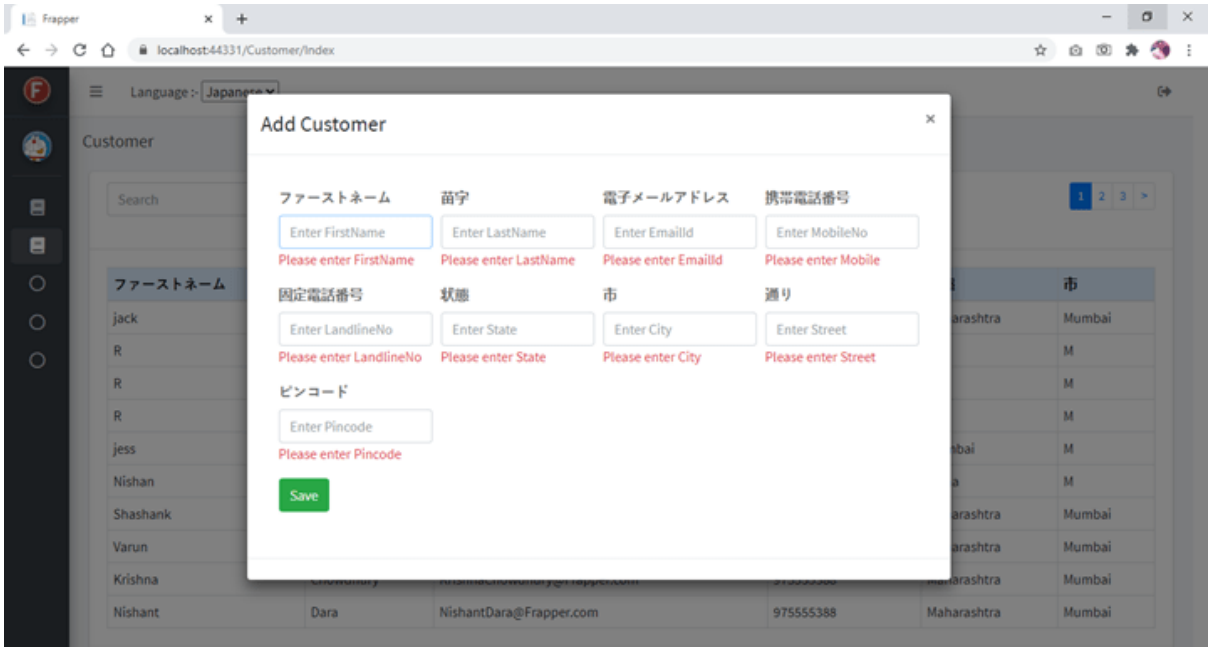



**Modal popups**

We have used bootstrap modal popups and we have also rendered form in modal popup. And wrote whole validation using jQuery validation plugin.



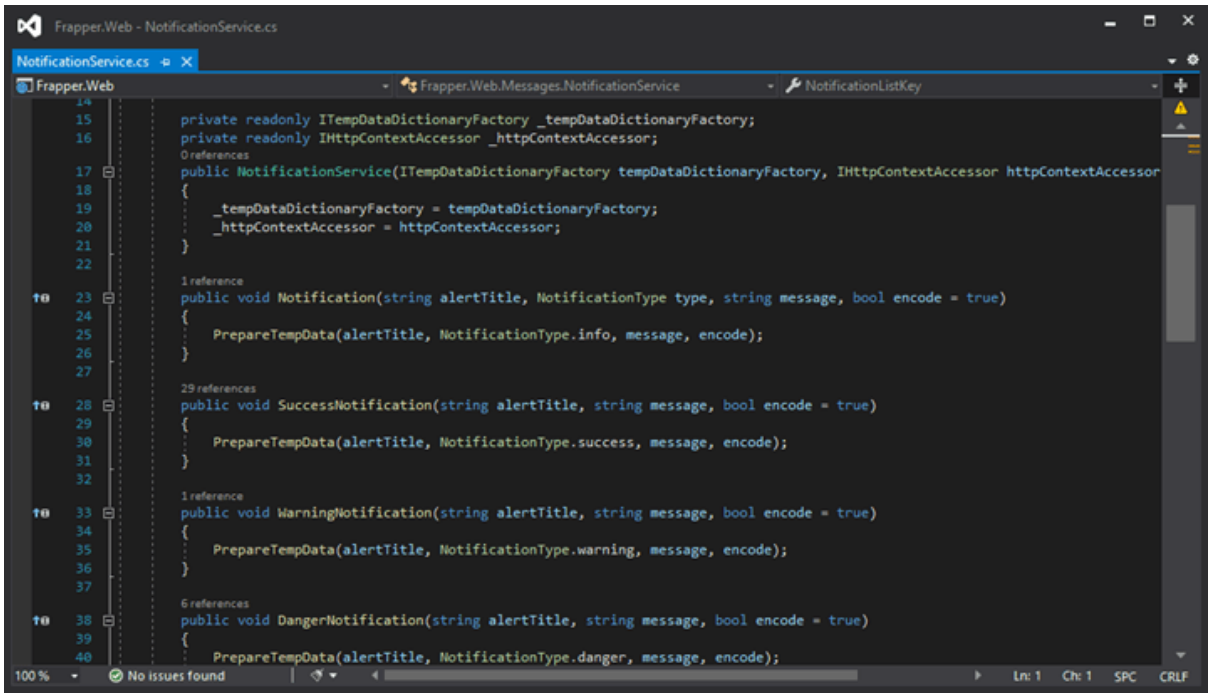
## Modal popups




## Notification

We are using Toasts Notification in this project.


Notification Service



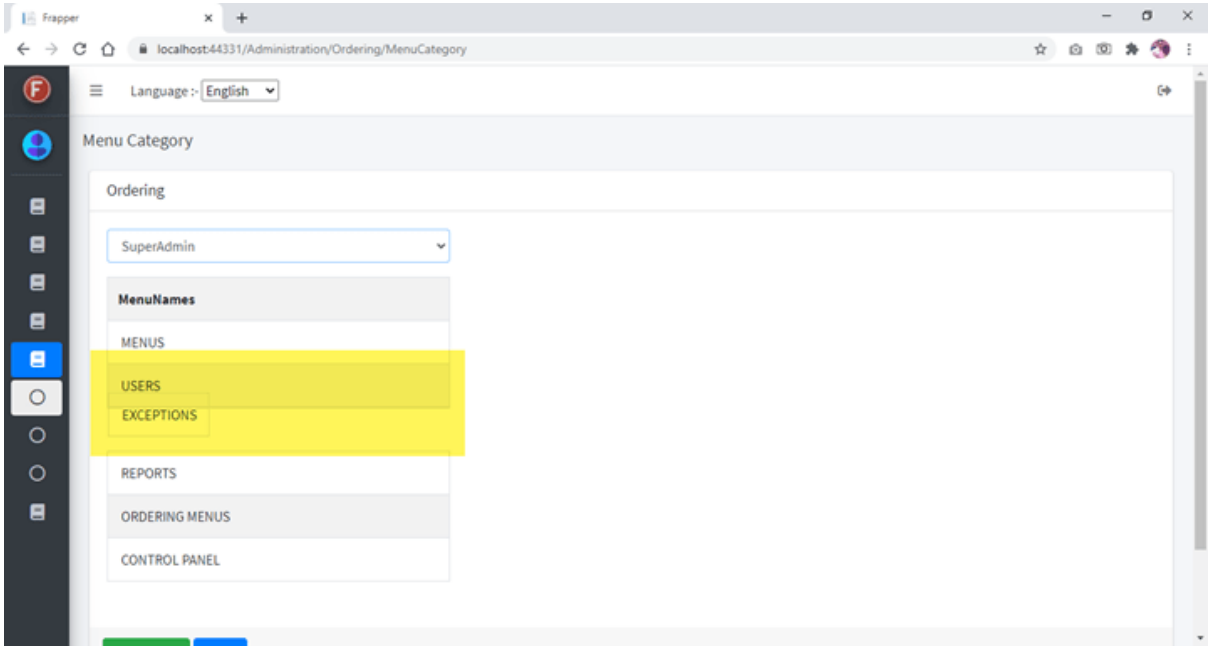
Notification Partial View

_BootstrapAlert.cshtml

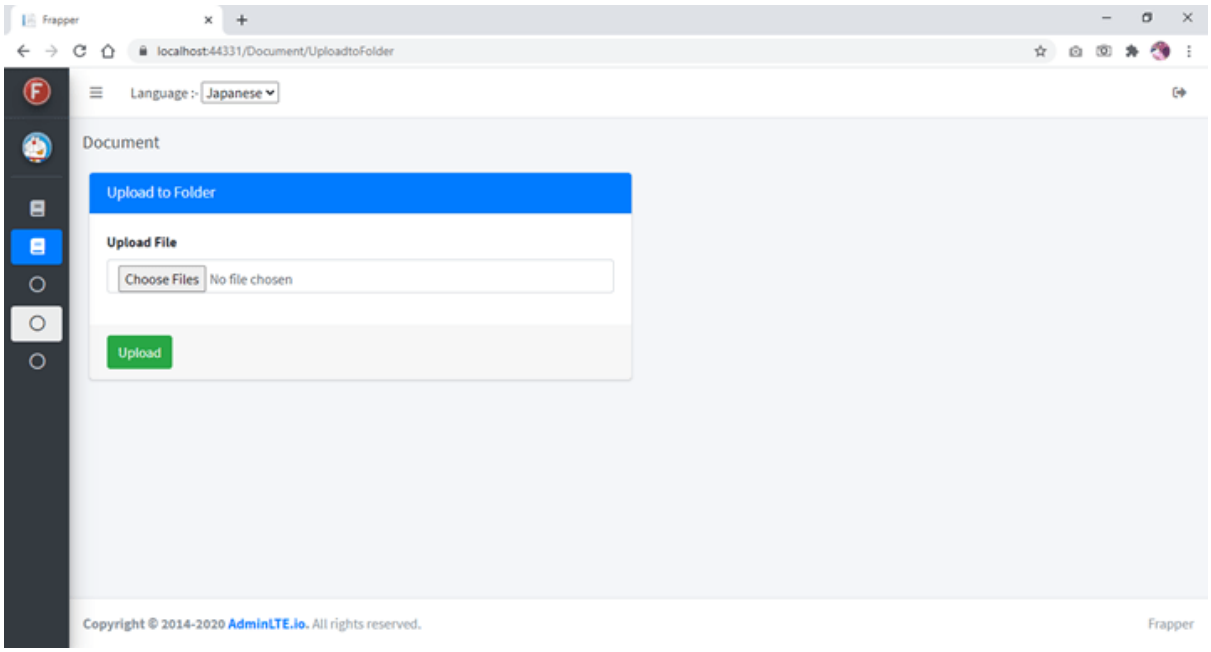
```
if (note.Type == NotificationType.success)
{
    <script>
        $(document).ready(function () {
            displayToastsSuccessNotifications('@Html.Raw(note.AlertTitle)','@Html.Raw(note.Message)');
        });
    </script>
}
if (note.Type == NotificationType.warning)
{
    <script>
        $(document).ready(function () {
            displayToastsWarningNotifications('@Html.Raw(note.AlertTitle)','@Html.Raw(note.Message)');
        });
    </script>
}
if (note.Type == NotificationType.info)
{
    <script>
        $(document).ready(function () {
            displayToastsInformationNotifications('@Html.Raw(note.AlertTitle)','@Html.Raw(note.Message)');
        });
    </script>
}

if (note.Type == NotificationType.danger)
{
    <script>
        $(document).ready(function () {
            displayToastsDangerNotifications('@Html.Raw(note.AlertTitle)','@Html.Raw(note.Message)');
        });
    </script>
```

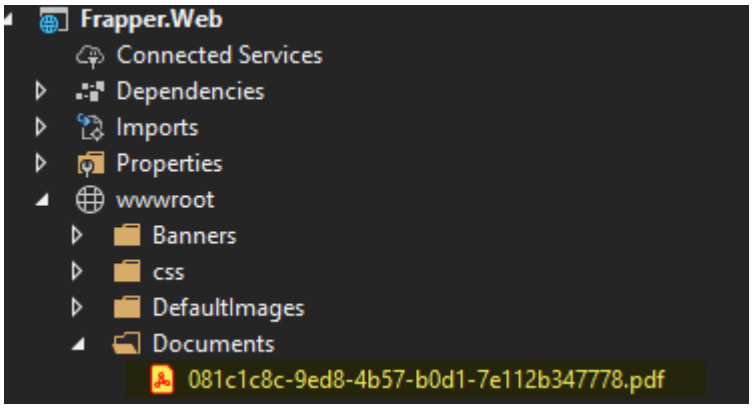
## Ordering Menus

For ordering menus, we are using jQuery UI sortable.

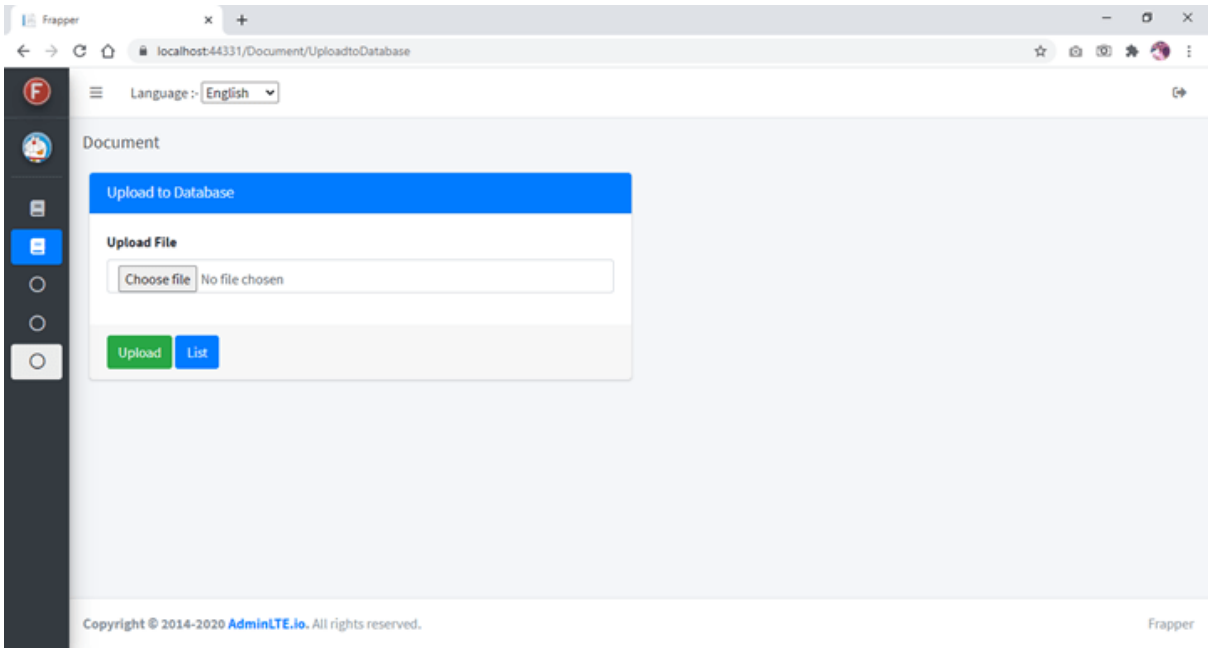


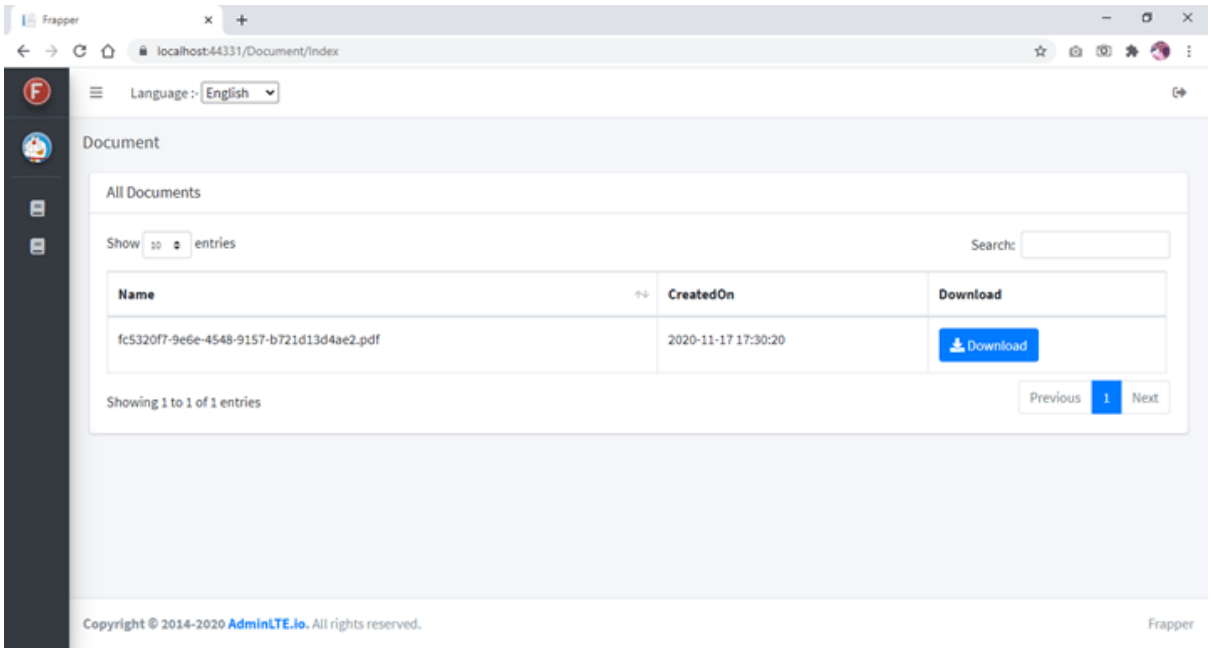
## Upload Files to store in a folder



## Upload Files to store in Database




**Reference** : https://github.com/saineshwar/Frapper